

Faculty of Engineering
Department of Informatics Engineering- Software
and Information Systems Engineering



Software Project Management Website
Applying Task Management in Software Projects

senior1 Project- Completed the requirements for obtaining a bachelor's degree
in Informatics Engineering - Software Engineering and Information Systems

Prepared by

Amjad Yousef Remy Alhadad Ajoud Al-Mashouh

Supervised By

Dr. Kadan Al-Jumaa

Eng. Farah Fares

2026

Supervisor Certification

I certify that the preparation of this project entitled "Software Project Management System"

Prepared by Amjad Yousef Remy Alhadad, was made under my supervision at the Department of Software and Information Systems Engineering in partial fulfillment of the requirements for the Bachelor's Degree in Software and Information Systems Engineering.

Dr. Kadan Al-Jumaa

Signature: _____

Date: _____

Eng. Farah Fares

Signature: _____

Date: _____

Abstract

This project focuses on developing a collaborative web-based platform designed to manage software projects for students and small development teams. Traditional project management tools are often costly and overly complex, while project information is frequently scattered across multiple applications, leading to confusion and reduced productivity. The goal of this project is to provide a free, centralized platform that consolidates task management, team collaboration, and project documentation in one accessible location. The system employs Kanban-style boards with drag-and-drop functionality to visualize workflow stages (To Do, In Progress, Done). It supports task-level collaboration through comments and file attachments, while role-based access control (Admin, Project Owner, Team Member, Guest) ensures structured permissions. In the second phase, the system was extended with two AI-powered microservices: a Task Risk Predictor that analyzes task signals (deadline proximity, assignee workload, activity patterns, and historical performance) to classify each task's delay risk as HIGH, MEDIUM, or LOW; and a Team Behavior Profiler that aggregates member activity data and generates behavioral profiles using Google Gemma 4 (31B) via the HuggingFace Inference API. Both microservices are built with FastAPI (Python) and communicate with the main Next.js application through secure API proxy routes. The project was developed using Next.js, React, MongoDB, Mongoose, and FastAPI, following an incremental delivery methodology. All 17 functional requirements were successfully implemented and verified through comprehensive testing, demonstrating that the system meets its core objectives of providing an intelligent, student-friendly project management solution at zero cost.

TABLE OF CONTENTS

Contents

Supervisor Certification	2
Abstract	3
TABLE OF CONTENTS	4
LIST OF FIGURES	6
Chapter 1 Introduction	7
1. Introduction	8
2. Problem Statement.....	8
3. Project Objective.....	8
4. Proposed System.....	9
5. Report Organization.....	9
6. Summary	9
Chapter 2: Fundamental Concepts and Literature Review	10
1. Introduction.....	11
2. Fundamental Concepts	11
3. Literature Review	12
4. Summary.....	16
Chapter 3: Project Management	17
1. Introduction.....	18
2. Project Charter	18
3. Statement of Work (SOW).....	19
4. Risk Management This section identifies potential risks that could impact project success and outlines mitigation strategies.	22
5. Gantt Chart	24
6. Summary.....	25
Chapter 4: System Analysis	26
1. Introduction.....	27
2. Project Timeline	27
3. Software Requirements Specification (SRS)	27
5. Initial Test Cases	45
7. Summary.....	51
Chapter 5: System Design	52
1. Introduction.....	53
2. System Architecture.....	53

3. Detailed Design for System Components	58
4. Database Schema Design	74
5. Summary.....	78
Chapter 6: Practical Implementation	79
1. Introduction.....	80
2. Used Tools and Technologies	80
3. System Interfaces	83
4. Test Cases Execution	91
Chapter 7: Report Overview.....	96
1. Introduction.....	97
2. Report Structure and Purposes	97
3. Summary.....	99

LIST OF FIGURES

Figure 1 3.1: Project Gantt Chart	24
Figure 2 4.2: Use Case Diagram	33
Figure 3 4.3: Login Sequence Diagram	39
Figure 4 4.4: Create Task Sequence Diagram	40
Figure 5 4.5: Drag-and-Drop Task Sequence Diagram	41
Figure 6 4.6: Class Diagram	42
Figure 7 4.7: ERD	44
Figure 8 5.1: System Architecture	53
Figure 9 5.2: Authentication Component Class Diagram	59
Figure 10 5.3: Project Management Component Class Diagram	61
Figure 11 5.4: Board Management Component Class Diagram	63
Figure 12 5.5: Task Management Component Class Diagram	65
Figure 13 5.6: Collaboration Component Class Diagram	67
Figure 14 Figure 5.8: Task Risk Predictor Class Diagram	71
Figure 15 Figure 5.9: Team Behavior Profiler Class Diagram	73
Figure 16 6.1: User Registration Page	83
Figure 17 6.3: Projects Dashboard	84
Figure 18 6.4: Landing/Marketing Page	85
Figure 19 6.5: Project Kanban Board	86
Figure 20 6.6: Task Details Modal	87
Figure 21 6.7: Share Project Modal	88
Figure 22 Figure 6.8: Task Risk Badge on Kanban Board	89
Figure 23 Figure 6.9: Team Behavior Profiler Interface	90

Chapter 1 Introduction

1. Introduction

This chapter introduces the core elements of the Software Project Management System, laying the foundation for understanding the problem it addresses, the project goals, the proposed system, the organization of the report, and a brief summary of key points.

2. Problem Statement

Students and small development teams face significant challenges when managing software projects. Existing tools like Trello, Jira, and Asana are often expensive and overly complex for academic needs. Project information is scattered across multiple platforms—chat apps for communication, cloud storage for files, and separate tools for task tracking—leading to confusion and lost information. Teams struggle to maintain a unified view of project status, resulting in unclear task ownership, missed deadlines, and reduced productivity. There is a clear need for a free, centralized, and student-friendly platform that consolidates project planning, task tracking, and collaboration in one place.

3. Project Objective

The main objective is to develop a free, web-based project management system tailored for students and small teams. Key goals include providing a centralized hub for tasks and documentation, implementing Kanban boards with drag-and-drop functionality for visual task management, enabling task-level comments and file attachments for better collaboration, supporting deadlines and notifications to improve accountability, and implementing role-based access control with four roles (Admin, Project Owner, Team Member, Guest) to manage permissions effectively. Additionally, the system integrates AI-powered microservices to provide proactive project insights: a Task Risk Predictor that automatically analyzes task signals and predicts delay probability, and a Team Behavior Profiler that generates behavioral analysis reports for each team member, enabling project owners to make informed decisions and intervene before issues escalate.

4. Proposed System

The Software Project Management System is a collaborative platform built around Kanban boards. Users can create projects, organize tasks into customizable columns (To Do, In Progress, Done), and drag tasks between columns to reflect workflow progress. Each task supports deadlines, comments, and file attachments, with automatic notifications to keep team members informed. The system uses a five-layer architecture (Next.js, API routes, business logic, repository layer, MongoDB) to ensure scalability and maintainability, with an intuitive interface designed specifically for straightforward student workflows. The system was further enhanced with two intelligent AI microservices built using FastAPI and Python. The Task Risk Predictor analyzes five signals per task — deadline proximity, column status relative to deadline, assignee workload, recent activity level, and historical on-time completion rate — and produces a risk classification (HIGH, MEDIUM, or LOW) with an AI-generated reason and actionable recommendation displayed as a badge on each task card. The Team Behavior Profiler aggregates per-member statistics across the project and uses Google Gemma 4 (31B) to generate natural language behavioral profiles covering work style, key strengths, and improvement suggestions for each team member. Both services implement graceful fallback mechanisms that return rule-based results if the AI model is unavailable, ensuring system reliability.

5. Report Organization

This report contains seven chapters:

Chapter 1 introduces the problem, objectives, and proposed system including the AI features added in the second phase.

Chapter 2 explains core concepts — including Large Language Models and microservices architecture — and compares existing tools.

Chapter 3 covers project planning, timeline, risk management, and the extended development phases.

Chapter 4 details all 17 system requirements, use cases including AI use cases (UC-06, UC-07), diagrams, test cases, and the Requirements Traceability Matrix.

Chapter 5 presents the system architecture including the AI microservices layer, and detailed component designs for all system modules.

Chapter 6 shows implementation details, screenshots including AI interfaces, tools and technologies, and complete test execution results for all 22 test cases.

Chapter 7 summarizes the report structure and overall project outcomes.

6. Summary

This chapter has outlined the need for a centralized, student-friendly project management system that addresses the limitations of expensive and fragmented tools. It explained the objectives of the project, introduced the proposed Kanban-based solution at a high level, and provided an overview of the report's structure. This introduction sets the stage for a deeper exploration of each aspect of the project in the following chapters.

Chapter 2: Fundamental Concepts and Literature Review

1. Introduction

This chapter provides a foundation for understanding the Software Project Management System by exploring fundamental concepts and reviewing related tools. It examines essential terminology, key principles behind the technologies used in the system, and presents an overview of existing project management solutions. The aim is to establish the theoretical background and compare similar systems to highlight the advantages and design decisions of the proposed platform.

2. Fundamental Concepts

This section explains the core concepts and principles that form the foundation of the proposed system.

Kanban Methodology

Kanban is a visual workflow management method that organizes work into boards, columns, and cards. Each board represents a project, columns represent workflow stages (such as To Do, In Progress, Done), and cards represent individual tasks. The core principle is visualizing work to improve transparency and tracking progress by moving task cards between columns as work advances. This system was adopted because it provides an intuitive way for student teams to manage project tasks without complex planning overhead, allowing teams to see the current status of all work items at a glance and identify bottlenecks when too many tasks accumulate in one stage.

Role-Based Access Control (RBAC)

Role-Based Access Control is a security model that assigns permissions based on user roles rather than individual users. In this system, four roles are defined: Admin (manages users and system settings), Project Owner (creates projects, manages boards and columns, invites team members), Team Member (creates and edits tasks, adds comments and attachments), and Guest (view-only access). This approach ensures that users can only perform actions appropriate to their responsibility level, protecting project data from accidental or unauthorized modifications while enabling efficient collaboration.

Drag-and-Drop Interface

Drag-and-drop is a user interaction technique where users click on an object, hold the mouse button, move the object to a new location, and release to drop it. In this system, users drag task cards between Kanban columns to update task status (for example, dragging a task from "To Do" to "In Progress"). This provides a faster and more intuitive alternative to opening forms or menus for status updates, making task management more efficient and visually clear.

Task Management and Workflow Automation

Task management involves organizing work items with attributes such as title, description, assignee, deadline, and status. Workflow automation executes predefined actions automatically when certain events occur—for example, sending notifications when a task deadline is approaching or when a task status changes. In this system, automation reduces the need for manual checking and ensures team members stay informed of important project updates without constant monitoring, improving accountability and reducing missed deadlines.

Large Language Models (LLMs)

Large Language Models are AI systems trained on massive text datasets, enabling them to understand natural language context and generate human-like text responses. These models learn statistical patterns across billions of parameters, allowing them to reason about problems, summarize information, and produce structured outputs such as JSON. In this system, Google Gemma 4 (31B parameters) is used to analyze task context and generate natural language risk assessments and behavioral profiles. The model is accessed via the HuggingFace Inference API with deterministic settings (temperature: 0.15) to ensure consistent and reliable outputs across repeated requests.

Microservices Architecture

Microservices is an architectural pattern where an application is structured as a collection of small, independently deployable services that communicate with each other via well-defined APIs. Unlike a monolithic architecture where all functionality resides in a single codebase, each microservice handles a single responsibility and can be developed, deployed, and scaled independently. This separation reduces the risk of cascading failures and allows different parts of the system to use different programming languages or frameworks. In this system, two Python/FastAPI microservices handle AI functionality — the Task Risk Predictor on port 8001 and the Team Behavior Profiler on port 8002 — while the main Next.js application acts as the orchestrator, forwarding requests through proxy API routes.

3. Literature Review

This section analyzes existing project management tools to understand the current landscape and identify gaps that the proposed system addresses. The comparison focuses on three popular platforms and evaluates their strengths, weaknesses, and suitability for student teams.

Trello

Trello is a visual collaboration tool built around Kanban boards, offering simple task management through cards, lists, and boards.

Advantages:

Intuitive drag-and-drop interface suitable for small teams and personal projects

Free plan includes unlimited cards, lists, and up to 10 boards per workspace

Easy to set up with minimal learning curve

Visual workflow management with customizable columns

Disadvantages:

Limited advanced features (Gantt charts, dependencies) in free tier

Basic collaboration tools—lacks built-in documentation and revision history

Paid plans required for automation and multiple board views

No role-based permissions in free version

Jira

Jira is a comprehensive project management platform designed for software development teams, offering advanced features like sprint planning, backlog management, and detailed reporting.

Advantages:

Powerful for Agile/Scrum workflows with extensive customization

Free plan supports up to 10 users with Scrum/Kanban boards

Advanced permissions and project roles in paid plans

Strong integration ecosystem

Disadvantages:

Steep learning curve—overly complex for simple student projects

Interface designed for enterprise teams, not beginners

Many essential features locked behind expensive paid plans

Requires significant configuration time before productive use

Asana

Asana is a work management platform offering multiple views (list, board, timeline, calendar) and task dependency tracking.

Advantages:

Multiple visualization options including Gantt charts

Good for managing complex workflows and dependencies

Clean, modern interface

Disadvantages:

Free tier has limited features and user restrictions

Paid plans start at \$9+ per user per month, expensive for students

Can feel overwhelming for teams needing simple task tracking

No built-in documentation or file revision history in basic plans

Comparative Analysis

The table below compares the proposed Software Project Management System with existing tools:

Table 2.1: Comparative Analysis of Project Management Tools

Feature	Our System	Trello	Jira	Asana
Free for students	✓ Unlimited	✓ Limited (10 boards)	✓ Limited (10 users)	✗ Very limited
Kanban boards	✓	✓	✓	✓
Drag-and-drop	✓	✓	✓	✓
Role-based access	✓	✗ (Paid only)	✓ (Paid)	✗ (Paid only)
Task comments	✓	✓ Basic	✓	✓
File attachments	✓	✓ Limited	✓	✓ Limited
Deadlines & reminders	✓	✓ (Paid)	✓	✓
Project documentation	Planned	✗	✗	✗
Ease of use	Simple	Simple	Complex	Moderate
Target audience	Students/small teams	General	Enterprise	General
AI Task Risk Prediction	✓	✗	✗	✗
AI Team Behavior Profiling	✓	✗	✗	✗

Our Advantage: The proposed system focuses on providing essential Kanban features with role-based collaboration at zero cost, specifically designed for student workflows file:///C:/Users/amjad/Downloads/Feature-OurSystem-Trello-Jira-Asana.csv without the complexity and pricing barriers of commercial tools.

4. Summary

This chapter presented the fundamental concepts underlying the Software Project Management System, including Kanban methodology for visual workflow management, Role-Based Access Control for secure collaboration, drag-and-drop interfaces for intuitive task updates, workflow automation for improved efficiency, Large Language Models for AI-powered natural language analysis, and microservices architecture for independent deployment of intelligent components. The literature review compared existing project management tools (Trello, Jira, Asana) and demonstrated that while these platforms offer powerful features, they often present cost barriers, complexity, or limited free-tier functionality for student teams — and none of them offer built-in AI risk prediction or team behavior analysis. The comparative analysis highlighted the proposed system's key differentiators: essential Kanban and collaboration features at zero cost, specifically tailored to student workflows, enhanced by AI-powered proactive project insights.

Chapter 3: Project Management

1. Introduction

This chapter outlines the project management approach adopted for the Software Project Management System, covering the project charter, timeline, risk management strategy, and visual project planning through a Gantt chart. Effective project management ensures that development activities remain organized, risks are identified and mitigated, and deliverables are completed on schedule.

2. Project Charter

The project charter formally authorizes the Software Project Management System and defines key project parameters, including objectives, stakeholders, roles, and high-level scope.

Project Overview

- ❖ Project Name: Software Project Management Website
- ❖ Project Duration: October 2025 – January 2026 (14 weeks)
- ❖ Prepared By: Amjad Yousef Remy Alhadad
- ❖ Supervised By: Dr. Rania Rajab, Eng. Mohammad Othman

Objective:

Develop a free, student-friendly collaborative platform for managing software projects using Kanban boards, task tracking, and role-based collaboration.

Roles and Responsibilities

Table 3.1: Roles and Responsibilities

Role Name	Responsibilities
Project Developer Amjad Yousef Remy Alhadad	System design, implementation, testing, documentation, and deployment.
Project Supervisor Dr. Rania Rajab	Academic supervision, project guidance, review of deliverables, and final approval.
Technical Supervisor Eng. Mohammad Othman	Technical guidance, architecture review, code review, and implementation support.

Project Scope

The system includes:

User authentication (registration, login, password recovery, logout)

Project management (create, view, delete projects)

Kanban board management (create/delete columns, add/edit/delete tasks, drag-and-drop functionality)

Task collaboration (comments, file attachments, deadlines)

Role-based access control (Admin, Project Owner, Team Member, Guest)

Project sharing and member invitation with role assignment

Notifications and reminders

AI-powered Task Risk Prediction (Task Risk Predictor microservice — FastAPI, port 8001)

AI-powered Team Behavior Profiling (Team Behavior Profiler microservice — FastAPI, port 8002)

Project templates (Blank, Software Sprint, Research, Marketing)

Next.js API proxy routes connecting the main application to AI microservices

Out of Scope (Future Work):

Revision history for task changes

Built-in chat module

Real-time live updates (WebSocket-based)

External integrations (GitHub, Google Drive, Calendar)

Two-factor authentication (2FA)

3. Statement of Work (SOW)

The Statement of Work defines the project phases, deliverables, and timeline.

Development Approach

The project follows an incremental delivery methodology, which ensures that a functional product is available early in the development cycle, with continuous enhancements added iteratively. This approach enables early user feedback, reduces development risks, and allows for adjustments based on testing and evaluation.

Project Phases

The development is structured into six phases spanning 14 weeks:

Phase 1: Foundation (2 weeks, Oct 14 – Oct 28, 2025)

Requirements gathering and analysis

Database design (MongoDB schema)

System architecture design (5-layer model)

Development environment setup

Deliverable: System architecture document, database schema, development environment ready

Phase 2: Authentication & Core Backend (3 weeks, Oct 28 – Nov 18, 2025)

User authentication system (register, login, password recovery)

Role-based access control implementation

User management API

Deliverable: Authentication system functional, user management API ready

Phase 3: Project Board Core (3 weeks, Nov 18 – Dec 9, 2025)

Project creation and management

Kanban board interface

Column management (add/delete columns)

Task creation and basic dashboard

Drag-and-drop functionality

Deliverable: MVP delivered—working Kanban board with basic task management

Phase 4: Enhanced Features & Collaboration (3 weeks, Dec 9 – Dec 30, 2025)

Task comments system

File attachment functionality

Task deadlines and status tracking

Notification system

Deliverable: Collaboration features complete, notification system operational

Phase 5: Frontend Polish & Optimization (2 weeks, Dec 30, 2025 – Jan 13, 2026)

UI/UX refinement

Dashboard enhancements and reports

Performance optimization

Extensive testing and bug fixes

User feedback gathering

Deliverable: Production-ready UI, performance optimized

Phase 6: Testing, Deployment & Launch (1 week, Jan 13 – Jan 20, 2026)

Final system testing

Deployment to production environment

User training and documentation

Official platform launch

Deliverable: System deployed and operational

4. Risk Management

This section identifies potential risks that could impact project success and outlines mitigation strategies.

Table 3.2: Risk Management

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
R1	Technical complexity in implementing drag-and-drop functionality	Medium	Medium	Use established libraries (e.g., dnd-kit, react-beautiful-dnd) with proven track record; allocate extra testing time.
R2	Database schema changes requiring data migration	Medium	High	Design flexible schema from the start; use MongoDB's schema-less advantages; implement versioning for schema changes.
R3	Timeline delays due to underestimated task complexity	Medium	High	Build buffer time into each phase; prioritize core features over optional enhancements; adopt incremental delivery to deliver MVP early.
R4	Authentication and security vulnerabilities	Low	High	Follow industry best practices (bcrypt for password hashing, JWT for sessions); conduct security testing; review authentication flow with technical supervisor.
R5	Scope creep from adding unplanned features	Medium	Medium	Maintain strict scope definition in project charter; defer non-essential features to "Future Work" section; focus on RTM-defined requirements only.
R6	Performance issues with large numbers of tasks/projects	Low	Medium	Implement pagination for project and task lists; optimize database

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
				queries with proper indexing; conduct performance testing during Phase 5.
R7	LLM API unavailability or rate limiting from HuggingFace/Novita provider	Medium	Medium	Implement rule-based fallback for both microservices; define timeout limits (10s); return deterministic results based on task signals alone when LLM is unreachable
R8	AI response latency impacting user experience on the Kanban board	Medium	Low	Execute AI risk calls asynchronously using the useTaskRisk hook; display loading skeleton states on task badges; cache results per session to avoid redundant API calls

5. Gantt Chart

The Gantt chart below visualizes the project timeline, showing the duration and dependencies of each phase from October 2025 to January 2026.

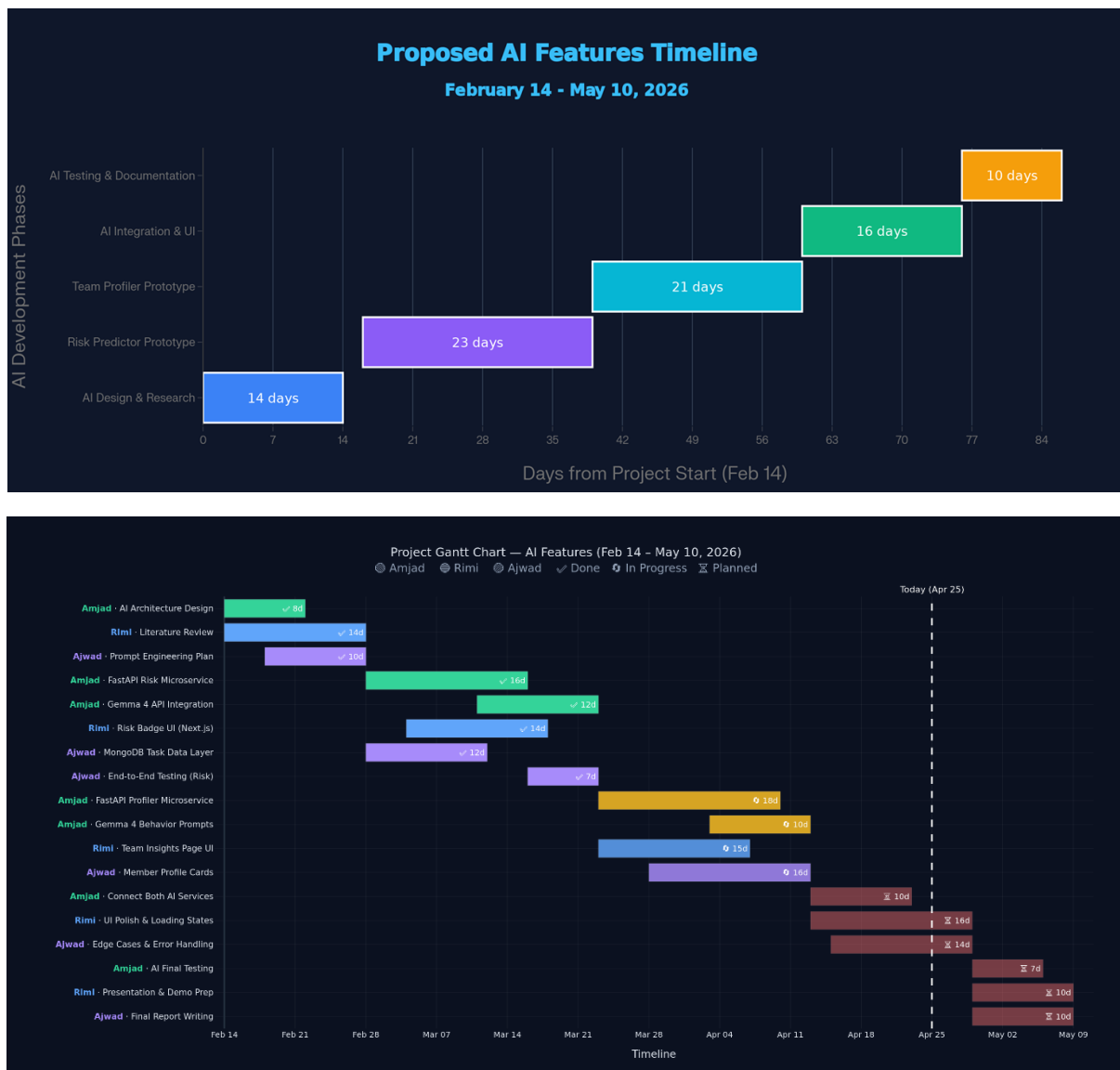


Figure 1 3.1: Project Gantt Chart

Key milestones:

- Oct 28, 2025: Foundation complete
- Nov 18, 2025: Authentication system ready
- Dec 9, 2025: MVP delivered (working Kanban board)
- Dec 30, 2025: Collaboration features complete
- Jan 13, 2026: Production UI ready
- Jan 20, 2026: Production launch

Senior Phase Extension (AI Development):

Following the successful completion of the Junior phase in January 2026, the project was extended by 12 weeks to incorporate AI-powered features. Two additional phases were added: Phase 7: AI Microservices Development (Feb 14 – Apr 14, 2026) — Design and implementation of the Task Risk Predictor (FastAPI :8001) and Team Behavior Profiler (FastAPI :8002), including rule engine development, LLM integration with Gemma 4, and prompt engineering. Phase 8: AI Integration & Testing (Apr 14 – May 10, 2026) — Integration of both microservices with the Next.js application via proxy API routes, implementation of graceful fallback mechanisms, UI components (RiskBadge, Team Insights page), and comprehensive AI test case execution.

6. Summary

This chapter presented the project management framework for the Software Project Management System, including the project charter with defined roles and scope, the Statement of Work outlining six development phases over 14 weeks, a risk management plan identifying potential threats and mitigation strategies, and a Gantt chart visualizing the project timeline. The incremental delivery approach ensures early availability of core functionality while allowing for continuous refinement based on testing and feedback.

Chapter 4: System Analysis

1. Introduction

This chapter provides a comprehensive analysis of the Software Project Management System, covering the project timeline, detailed requirements specification, system modeling through use cases and diagrams, initial test cases, and the Requirements Traceability Matrix (RTM). System analysis is critical for ensuring that all stakeholder needs are clearly defined, documented, and traceable throughout the development lifecycle.

2. Project Timeline

The project follows a 14-week incremental development timeline from October 2025 to January 2026, structured into six distinct phases. Each phase builds upon the previous one, delivering functional increments that can be tested and validated before proceeding to subsequent phases. The timeline emphasizes early delivery of core functionality (MVP by Phase 3) to enable user feedback and iterative improvements.

3. Software Requirements Specification (SRS)

The Software Requirements Specification document defines all functional and non-functional requirements for the system.

3.1 Functional Requirements

Functional requirements describe what the system must do—the specific behaviors, features, and operations that users can perform.

FR-01: User Registration

Description: The system shall allow new users to create accounts by providing necessary information.

Inputs: Full name, email address, password

Process: Validate email format, check for duplicate email, hash password, create user record with default role

Outputs: User account created, success message displayed, redirect to dashboard or login

FR-02: User Login

Description: The system shall provide secure authentication for registered users.

Inputs: Email address, password

Process: Validate credentials against database, create session token (JWT)

Outputs: User redirected to dashboard, session established

FR-03: Password Recovery

Description: The system shall allow users to recover access if they forget their password.

Inputs: Registered email address

Process: Verify email exists, send password reset link/instructions

Outputs: Password reset email sent, success message confirmed

FR-04: Create Project

Description: The system shall allow authenticated users to create new project boards.

Inputs: Project title, description (optional)

Process: Validate inputs, create project record with user as owner, initialize default columns

Outputs: New project appears in project list with correct metadata

FR-05: View Project List

Description: The system shall display all projects owned by or shared with the current user.

Inputs: User session

Process: Query database for projects where user is owner or member

Outputs: List of projects with title, owner, created date displayed on dashboard

FR-06: Delete Project

Description: The system shall allow project owners to delete projects.

Inputs: Project ID, confirmation

Process: Verify user is project owner, delete project and all associated data (columns, tasks, comments, attachments)

Outputs: Project removed from system, no longer accessible to any user

FR-07: Manage Columns

Description: The system shall allow users to create and delete columns within a project board.

Inputs: Column name (for creation), column ID (for deletion)

Process: Create column record linked to project, or delete column and reassign/remove tasks

Outputs: Column appears on board (creation), or column removed with board layout updated (deletion)

FR-08: Create Task

Description: The system shall allow users to create task cards inside columns.

Inputs: Task title, column ID

Process: Validate inputs, create task record linked to column

Outputs: Task card appears in the selected column

FR-09: Drag-and-Drop Task

Description: The system shall allow users to reorder tasks and move them between columns using drag-and-drop.

Inputs: Task ID, source column, target column, new position

Process: Update task's column ID and position, persist changes to database

Outputs: Task displayed in new column, position persists after page refresh

FR-10: Set Task Deadline

Description: The system shall allow users to set and update task deadlines.

Inputs: Task ID, deadline date

Process: Validate date is in future, update task record

Outputs: Deadline saved and displayed, status badge shows deadline info

FR-11: Add Task Comment

Description: The system shall allow users to add and view comments on tasks.

Inputs: Task ID, comment text

Process: Create comment record linked to task and user

Outputs: Comment appears with user name, timestamp, and content

FR-12: Attach Files to Task

Description: The system shall allow users to attach files to tasks.

Inputs: Task ID, file (upload)

Process: Upload file to storage, create attachment record linked to task

Outputs: Attachment entry created with file name and uploader, file is downloadable

FR-13: Share Project

Description: The system shall allow project owners to share projects with team members and assign roles.

Inputs: Project ID, member email, role (Admin/Member/Guest)

Process: Verify email exists, create project member record with assigned role, send invitation notification

Outputs: Team member added to project with specified permissions

FR-14: Enforce Role-Based Access

Description: The system shall enforce role-based permissions for all operations.

Roles:

Admin: Manages users, edits tasks, manages columns, invites members

Project Owner: Full control—manages members, deletes project

Team Member: Creates/edits tasks, adds comments

Guest: View-only access

FR-15: Logout

Description: The system shall provide logout functionality and protect private pages.

Process: Destroy user session, redirect to login page

Outputs: User logged out, session terminated

FR-16: Task Risk Prediction

Description: The system shall provide AI-powered risk assessment for each task on the Kanban board. Inputs: Task data including title, deadline date, current column, assignee ID, number of open tasks assigned to the same user, days since last comment or column move, and historical on-time completion rate of the assignee.

Process: The Next.js proxy route (/api/task-risk) aggregates task signals and forwards them to the Task Risk Predictor microservice (FastAPI :8001).

A deterministic rule engine computes a pre-score based on five weighted signals. The Gemma 4 LLM enriches the result with contextual reasoning. The final score is merged and clamped to produce the risk level.

Outputs: Risk level classification (HIGH: score 60–100, MEDIUM: score 30–59, LOW: score 0–29), a confidence percentage, an AI-generated reason explaining the risk, and an actionable recommendation. Results are displayed as a color-coded badge on each task card (red/yellow/green).

FR-17: Team Behavior Profiling

Description: The system shall generate AI-powered behavioral profiles for each project member based on their activity history.

Inputs: Member user ID and project ID.

Process: The Next.js proxy route (/api/team-profile) aggregates per-member statistics from MongoDB collections (Tasks, Comments, Attachments): total tasks assigned, completed on time, completed late, average completion days, in-progress count, days since last activity, comments written, and files attached.

These statistics are sent to the Team Behavior Profiler microservice (FastAPI :8002), where Gemma 4 generates a structured profile.

Outputs: For each member — a work style classification, a list of key strengths, an actionable improvement suggestion, and a natural language performance summary.

Results are displayed as profile cards on the Team Insights page.

3.2 Non-Functional Requirements

Non-functional requirements define system qualities such as performance, security, usability, and scalability.

NFR-01: Performance

The system shall load the dashboard within 2 seconds under normal network conditions. Task drag-and-drop operations shall provide immediate visual feedback with less than 200ms latency.

NFR-02: Security

The system shall hash all passwords using bcrypt before storage. All API endpoints requiring authentication shall validate JWT tokens. The system shall protect against common vulnerabilities (SQL injection, XSS, CSRF).

NFR-03: Usability

The user interface shall be intuitive for users with basic computer skills and no prior project management tool experience. The system shall provide clear error messages and validation feedback.

NFR-04: Scalability

The system architecture shall support at least 100 concurrent users without performance degradation. The database design shall accommodate projects with up to 1000 tasks per board.

NFR-05: Availability

The system shall maintain 95% uptime during business hours, with scheduled maintenance windows communicated in advance.

NFR-06: Maintainability

The codebase shall follow modular architecture (5-layer model) with clear separation of concerns to facilitate future enhancements. Code shall include comments for complex logic and API documentation.

4. Requirements Modeling

This section presents visual models that capture system behavior and structure.

4.1 Use Case Diagram

The use case diagram provides a high-level view of system functionality and user interactions.

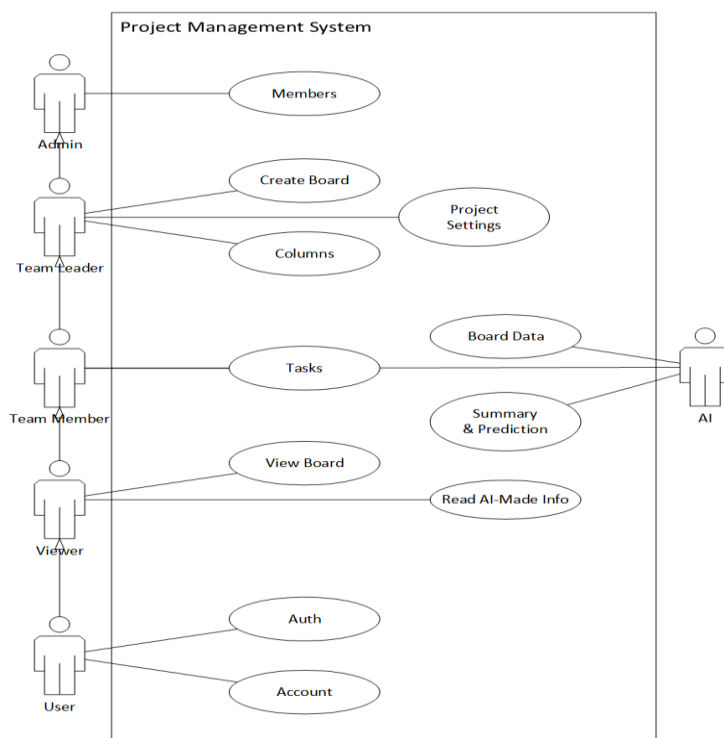


Figure 2 4.2: Use Case Diagram

Actors:

Admin: System administrator with elevated privileges

Project Owner: User who creates and owns projects

Team Member: Collaborator with task management permissions

Guest: Read-only observer

4.2 Use Case Specifications

Detailed specifications for key use cases are provided below.

UC-01: User Login

Attribute	Description
Use Case ID	UC-01
Use Case Name	User Login
Actor	All users (Admin, Project Owner, Team Member, Guest)
Precondition	User has a registered account
Postcondition	User is authenticated and redirected to dashboard
Main Flow	1. User navigates to login page 2. User enters email and password 3. System validates credentials 4. System creates session token 5. User redirected to dashboard
Alternative Flow 1	Invalid credentials: System displays error message "Invalid email or password"
Alternative Flow 2	Empty fields: System displays error message "Email and password required"

UC-02: Create Project

Attribute	Description
Use Case ID	UC-02
Use Case Name	Create Project
Actor	Project Owner, Admin
Precondition	User is logged in
Postcondition	New project created and appears in project list
Main Flow	1. User navigates to dashboard 2. User clicks "New Project" 3. User enters project title and description 4. User clicks "Create Project" 5. System validates inputs 6. System creates project with default columns 7. New project appears in list
Alternative Flow	Empty title: System displays error message "Project title is required"

UC-03: Drag-and-Drop Task

Attribute	Description
Use Case ID	UC-03
Use Case Name	Drag-and-Drop Task
Actor	Project Owner, Admin, Team Member
Precondition	User has access to project board, at least 2 columns exist, at least 1 task exists
Postcondition	Task moved to new column, change persisted in database
Main Flow	1. User opens project board 2. User clicks and holds task card 3. User drags task to target column 4. User releases mouse button 5. System updates task column and position 6. Task appears in new column 7. User refreshes page 8. Task remains in new column (persistence verified)

UC-04: Add Comment to Task

Attribute	Description
Use Case ID	UC-04
Use Case Name	Add Comment to Task
Actor	Project Owner, Admin, Team Member
Precondition	User has access to project, task exists
Postcondition	Comment added to task
Main Flow	1. User opens task details modal 2. User enters comment text 3. User clicks "Submit" 4. System validates input 5. System creates comment record 6. Comment appears with username and timestamp
Alternative Flow	Empty comment: System displays error message "Comment cannot be empty"

UC-05: Share Project

Attribute	Description
Use Case ID	UC-05
Use Case Name	Share Project
Actor	Project Owner, Admin
Precondition	User owns the project
Postcondition	Team member added with assigned role
Main Flow	1. User opens project 2. User clicks "Share" 3. User enters member email 4. User selects role (Admin/Member/Guest) 5. User clicks "Invite" 6. System verifies email exists 7. System creates project member record 8. Member appears in team list with assigned role

Attribute	Description
Alternative Flow	Email not found: System displays error message "User not found"

UC-06: View Task Risk Assessment

Attribute	Description
Use Case ID	UC-06
Use Case Name	View Task Risk Assessment
Actor	Project Owner, Admin, Team Member
Precondition	User has access to the project board; task exists with at least one of: deadline set, assignee assigned
Postcondition	Risk badge displayed on task card with level and AI-generated reason
Main Flow	1. User opens the project Kanban board 2. System calls /api/task-risk with task signals for each visible task 3. Next.js proxy forwards request to Task Risk Predictor (FastAPI :8001) 4. Rule engine computes pre-score from 5 signals 5. Gemma 4 LLM generates contextual reason and recommendation 6. Merged result returned to client 7. Color-coded risk badge (HIGH/MEDIUM/LOW) displayed on task card 8. User hovers over badge to see AI-generated reason and recommendation
Alternative Flow 1	LLM unavailable: Next.js proxy returns rule-based fallback result using deadline proximity signal only; badge still displayed
Alternative Flow 2	Task has no deadline or assignee: LOW risk returned by default with note "Insufficient data for full analysis"

UC-07: View Team Behavior Profiles

Attribute	Description
Use Case ID	UC-07
Use Case Name	View Team Behavior Profiles
Actor	Project Owner, Admin
Precondition	User owns or administers the project; at least one member has task activity recorded
Postcondition	AI-generated behavioral profile cards displayed for each project member
Main Flow	1. User navigates to the Team Insights page for a project 2. System aggregates per-member statistics from MongoDB (tasks, comments, attachments) 3. POST request sent to /api/team-profile with aggregated data 4. Next.js proxy forwards to Team Behavior Profiler (FastAPI :8002) 5. Gemma 4 generates structured profile for each member 6. Profile cards displayed with: work style classification, strengths, improvement suggestion, and performance summary
Alternative Flow 1	LLM unavailable: Rule-based fallback applied — on-time rate ≥80% → "Reliable performer", 50–79% → "Steady contributor", <50% → "Needs additional support"
Alternative Flow 2	Member has no task history: Profile card shows "Insufficient activity data" message

4.3 Sequence Diagrams

Sequence diagrams illustrate the interaction between system components over time for specific use cases.

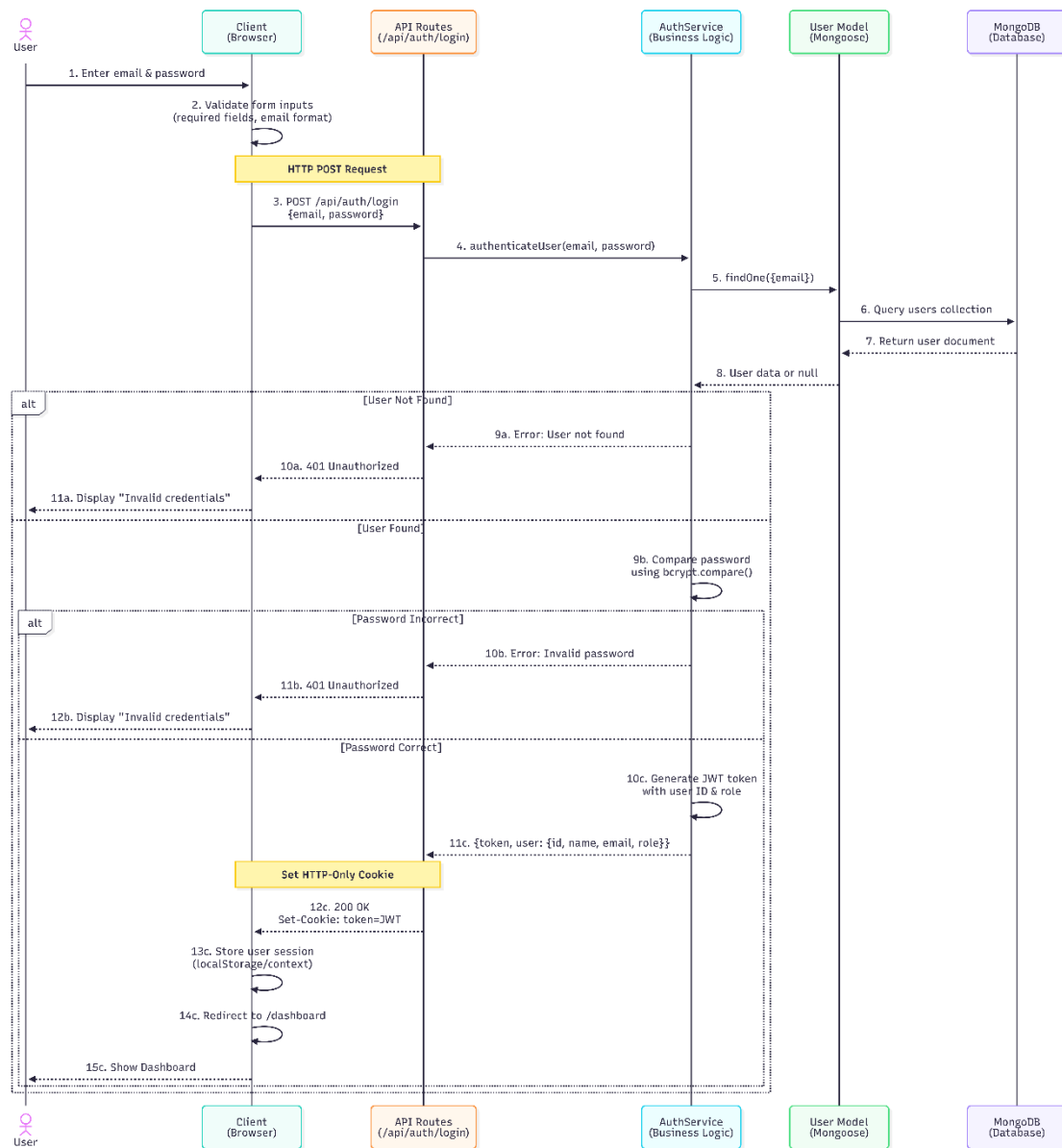


Figure 3 4.3: Login Sequence Diagram

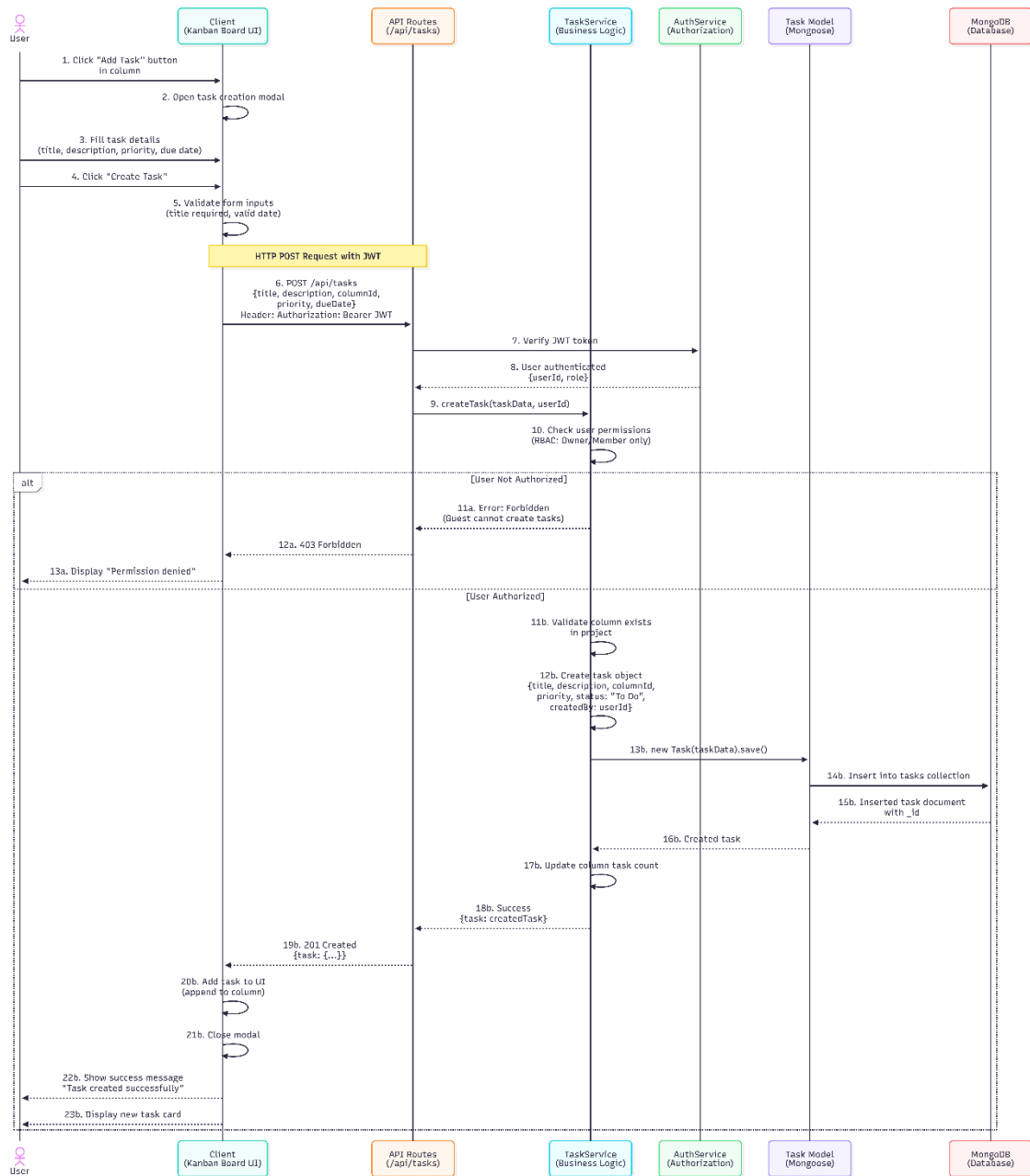


Figure 4 4.4: Create Task Sequence Diagram

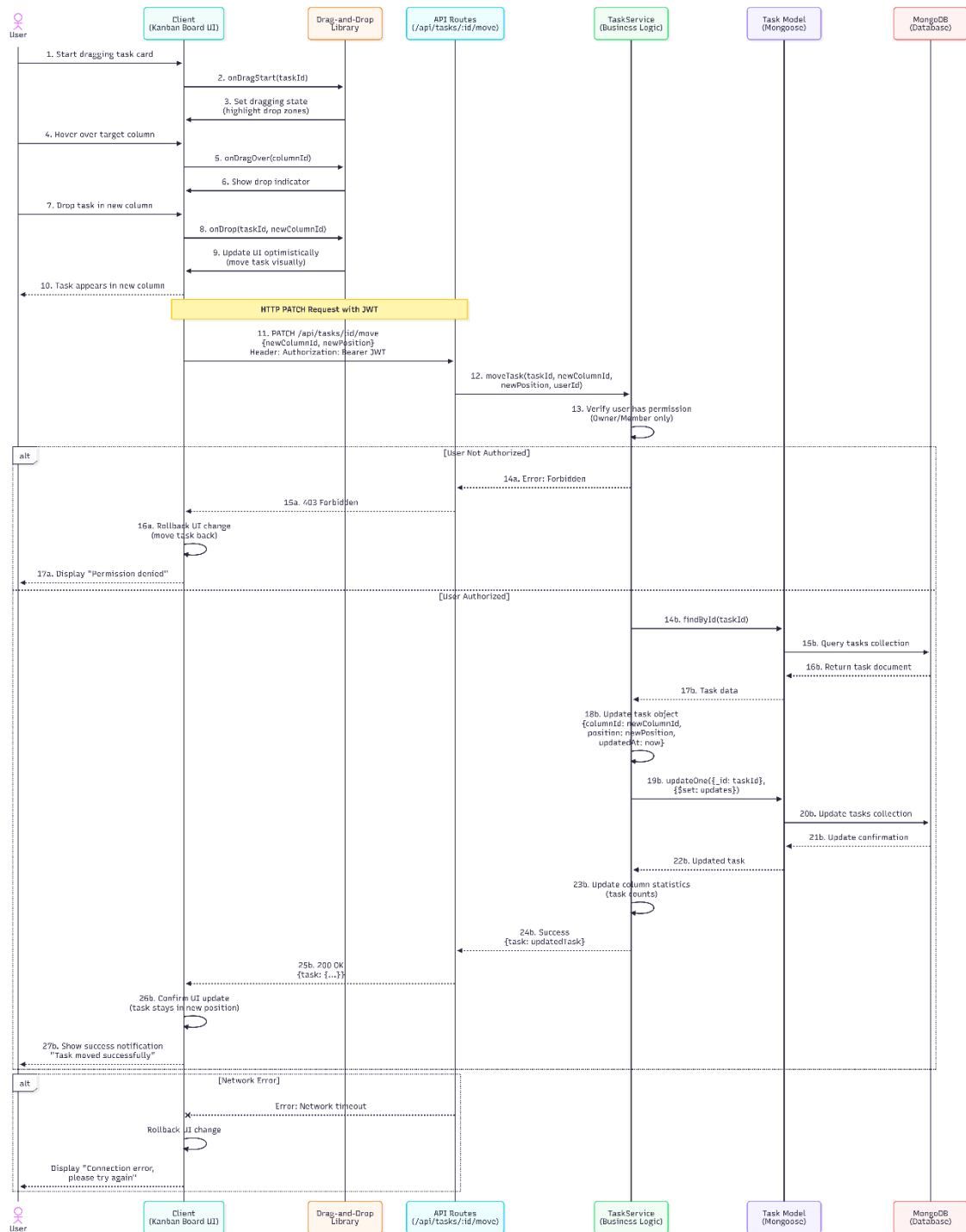


Figure 5 4.5: Drag-and-Drop Task Sequence Diagram

4.4 Class Diagram

The class diagram defines the system's object-oriented structure, showing entities, attributes, methods, and relationships.

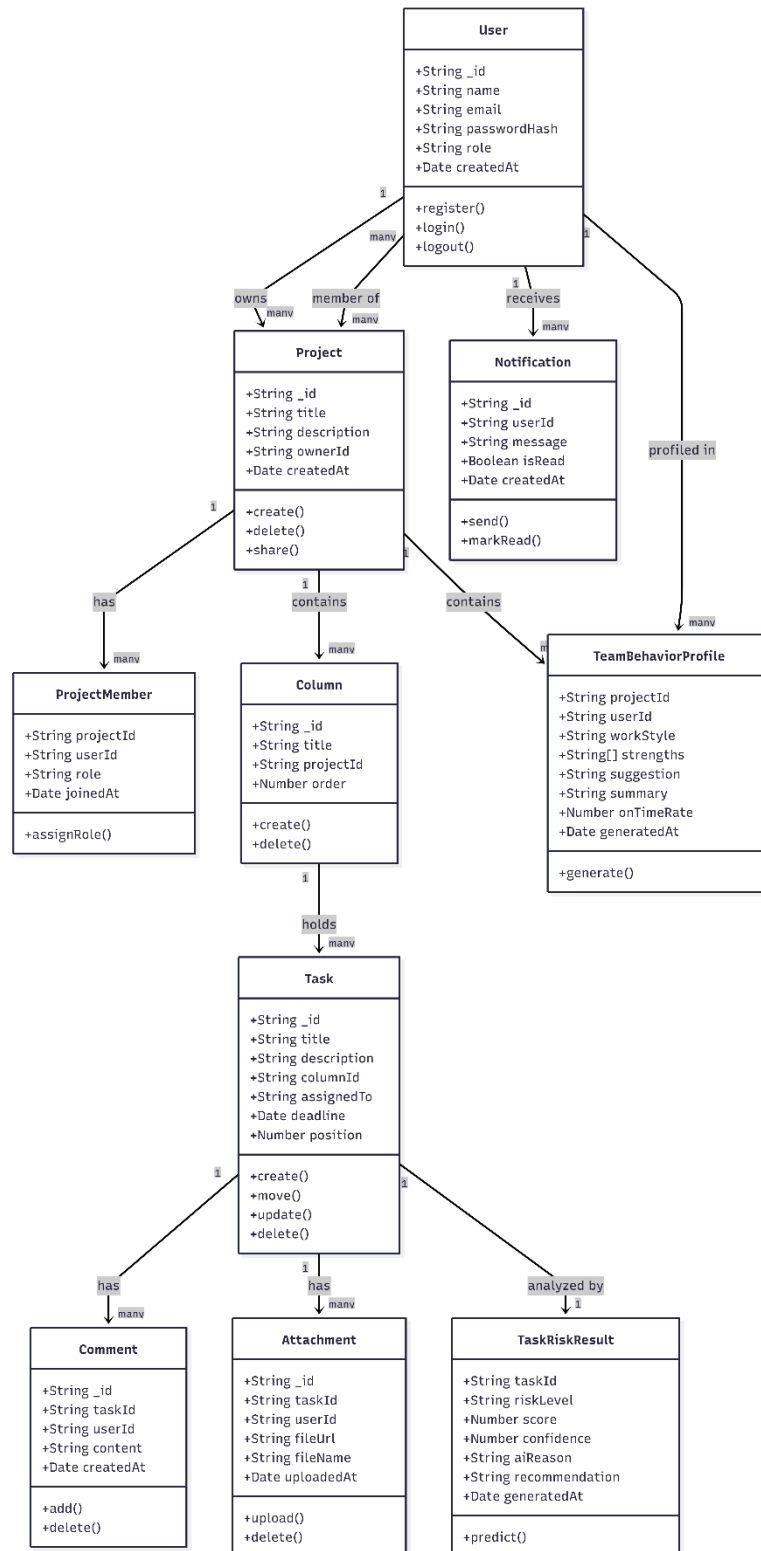


Figure 6 4.6: Class Diagram

Key Classes:

User: Base class with attributes (id, fullName, email, passwordHash, role, createdAt)

Admin, ProjectOwner, TeamMember, Guest: Inherit from User

Project: Contains columns, owned by ProjectOwner (id, ownerId, name, description, columns[], createdAt)

Column: Contains tasks (id, projectId, name, position, tasks[])

Task: Work item (id, columnId, title, description, assignedTo, deadline, status, createdAt)

Comment: Task discussion (id, taskId, userId, content, createdAt)

Attachment: File linked to task (id, taskId, uploadedBy, fileUrl, createdAt)

ProjectMember: Maps users to projects with roles (id, projectId, userId, memberRole, joinedAt)

4.5 Entity-Relationship Diagram (ERD)

The ERD shows the database schema designed for MongoDB, highlighting document-oriented collections and their relationships.

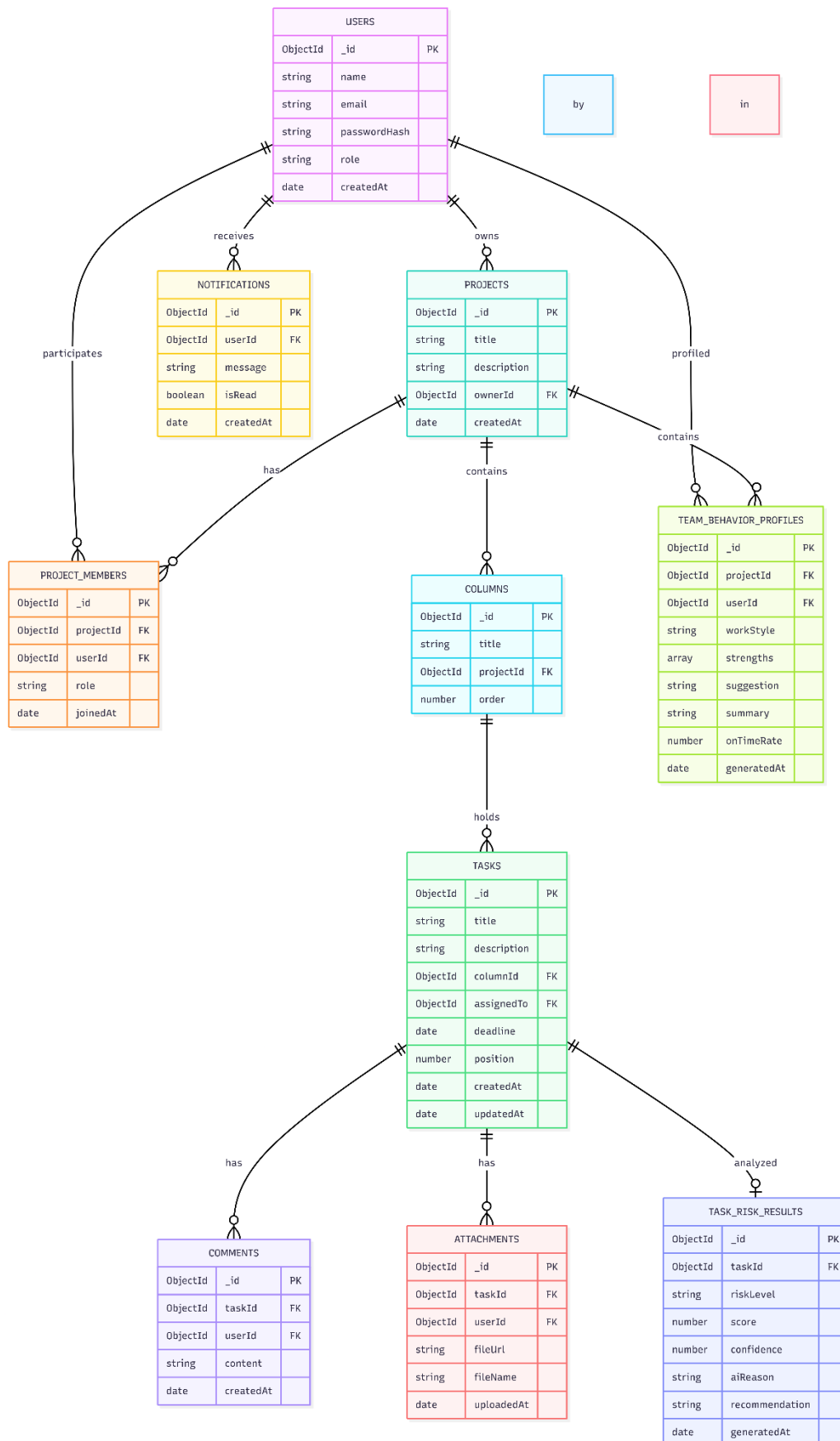


Figure 7 4.7: ERD

Key Relationships:

Users own Projects (1:N)

Projects contain Columns (1:N, embedded)

Columns contain Tasks (1:N)

Tasks have Comments (1:N)

Tasks have Attachments (1:N)

Users belong to Projects through ProjectMembers (M:N)

5. Initial Test Cases

Test cases verify that each functional requirement is implemented correctly.

Table 4.1: Authentication Test Cases

Table 4.1: Authentication Test Cases

TC ID	Test Case Title	Precondition	Steps	Expected Result
TC01	Login - Successful	User account exists	1. Open login page 2. Enter valid email/password 3. Click Login	User navigates to Dashboard, session created
TC02	Login - Wrong Password	User account exists	1. Open login page 2. Valid email, wrong password 3. Click Login	Error message "Invalid credentials" displayed, stay on login page
TC03	Register	Email not previously registered	1. Open register page 2. Fill form (email, password, name) 3. Submit	New user account created, success message displayed, redirect to login or Dashboard
TC04	Forgot Password	Email exists in system	1. Open Forgot Password page 2. Enter registered email 3. Submit	Password reset email/instruction sent, success message confirmed

Table 4.2: Project Management Test Cases*Table 4.2: Project Management Test Cases*

TC ID	Test Case Title	Precondition	Steps	Expected Result
TC05	Create Project	User logged in	1. Navigate to Dashboard 2. Click New Project 3. Enter title, description 4. Save	New project appears in project list with correct metadata
TC06	List Projects	User has owned/shared projects	1. Login 2. Open Dashboard	All owned and shared projects displayed with title, owner, created date
TC07	Delete Project	User is ProjectOwner	1. Dashboard → Select project 2. Click Delete 3. Confirm	Project removed from system, no longer accessible to any user

Table 4.3: Board and Task Management Test Cases*Table 4.3: Board and Task Management Test Cases*

TC ID	Test Case Title	Precondition	Steps	Expected Result
TC08	Add Column	User has access to project board	1. Open project board 2. Click Add Column 3. Enter column name 4. Save	New column appears on board, starts empty
TC09	Delete Empty Column	Empty column exists	1. Open project board 2. Select column 3. Delete 4. Confirm	Column removed, board layout updated accordingly
TC10	Add Task	At least one column exists	1. In column, click Add Task 2. Enter task title 3. Save	Task card appears in the selected column
TC11	Drag-Drop Task	2 columns, one task present	1. Drag task from column A to column B 2. Drop 3. Refresh page	Task now displayed in column B, position persists after refresh
TC12	Set Task Deadline	Task exists	1. Open Task Details 2. Set future date as deadline 3. Save	Deadline saved and displayed, status badge shows deadline info

Table 4.4: Collaboration Test Cases*Table 4.4: Collaboration Test Cases*

TC ID	Test Case Title	Precondition	Steps	Expected Result
TC13	Add Comment	Task exists	1. Open Task Details 2. Enter comment text 3. Submit	Comment appears with user name, timestamp, and content
TC14	Add Attachment	Task exists, file selected	1. Open Task Details 2. Click Add Attachment 3. Upload file	Attachment entry created with file name/uploader, file is downloadable
TC15	Share Project - Valid Email	User owns project, invitee exists	1. Open project 2. Click Share 3. Enter email, select role 4. Invite	Member added to project with specified role
TC16	Share Project - Invalid Email	User owns project	1. Open project 2. Click Share 3. Enter non-existent email 4. Invite	Error message "User not found" displayed
TC17	Logout	User logged in	1. Click Logout	User logged out, redirected to login page, session terminated

Table 4.5: AI Features Test Cases

TC ID	Test Case Title	Precondition	Steps	Expected Result
TC18	Risk Badge — HIGH Risk Task	Task is overdue AND assignee has ≥ 8 open tasks	1. Open project board 2. Observe task with past deadline	RED "HIGH" badge displayed on task card
TC19	Risk Badge — LOW Risk Task	Task has deadline ≥ 30 days away, assignee has < 3 open tasks	1. Open project board 2. Observe task with distant deadline	GREEN "LOW" badge displayed on task card
TC20	Risk Badge — LLM Fallback	LLM API is offline/unavailable	1. Disable network to AI service 2. Open project board	Rule-based badge still displayed; system does not crash; no error shown to user
TC21	Team Profile — Valid Generation	Project has ≥ 1 member with completed tasks and deadline data	1. Navigate to Team Insights page 2. Wait for profile generation	Profile cards displayed per member with: work style, strengths, suggestion, and performance summary
TC22	Team Profile — Fallback	LLM API offline during profile generation	1. Disable network to profiler service 2. Open Team Insights	Rule-based fallback profiles displayed based on on-time rate; no crash

6. Initial Requirement Traceability Matrix (RTM)

The RTM links each functional requirement to its corresponding use case, design component, and test case, ensuring complete coverage.

Table 4.5: Requirements Traceability Matrix

Table 4.5: Requirements Traceability Matrix

Req ID	Requirement	Related Use Case	Test Cases	Status
FR-01	User registration	UC-Register	TC03	Defined
FR-02	User login	UC-Login	TC01, TC02	Defined
FR-03	Password recovery	UC-ForgotPassword	TC04	Defined
FR-04	Create project	UC-ManageProjects	TC05	Defined
FR-05	View project list	UC-ViewDashboard	TC06	Defined
FR-06	Delete project	UC-ManageProjects	TC07	Defined
FR-07	Manage columns	UC-ManageColumns	TC08, TC09	Defined
FR-08	Create tasks	UC-ManageTasks	TC10	Defined
FR-09	Drag-drop tasks	UC-ManageTasks	TC11	Defined
FR-10	Set task deadline	UC-ManageTasks	TC12	Defined
FR-11	Add comments	UC-TaskCollaboration	TC13	Defined
FR-12	Attach files	UC-TaskCollaboration	TC14	Defined
FR-13	Share project	UC-ShareProject	TC15, TC16	Defined
FR-14	Role-based access	UC-AccessControl	TC15, TC16	Defined
FR-15	Logout	UC-SessionManagement	TC17	Defined
FR_16	Task Risk Prediction	UC-06	TC18, TC19, TC20	Defined
FR_17	Team Behavior Profiling	UC-07	TC21, TC22	Defined

7. Summary

This chapter provided a comprehensive system analysis for the Software Project Management System, including the project timeline, detailed functional and non-functional requirements covering all 17 functional requirements (FR-01 to FR-17) including the two new AI-powered features, requirements modeling through use case diagrams, detailed use case specifications for seven use cases (UC-01 to UC-07) including AI-specific use cases for Task Risk Assessment and Team Behavior Profiling, sequence diagrams, class diagram, and Entity-Relationship Diagram. Initial test cases were defined covering authentication, project management, board and task operations, collaboration features, and AI microservice functionality (TC01–TC22), along with the Requirements Traceability Matrix linking all requirements to use cases and test cases. These artifacts form the complete foundation for system design and implementation phases.

Chapter 5: System Design

1. Introduction

This chapter presents the system design for the Software Project Management System, covering the overall system architecture and detailed design for each major component. System design translates the requirements defined in Chapter 4 into a technical blueprint that guides implementation, ensuring scalability, maintainability, and clear separation of concerns.

2. System Architecture

The Software Project Management System employs a modern five-layer client-server architecture designed for scalability, maintainability, and clear separation of responsibilities.

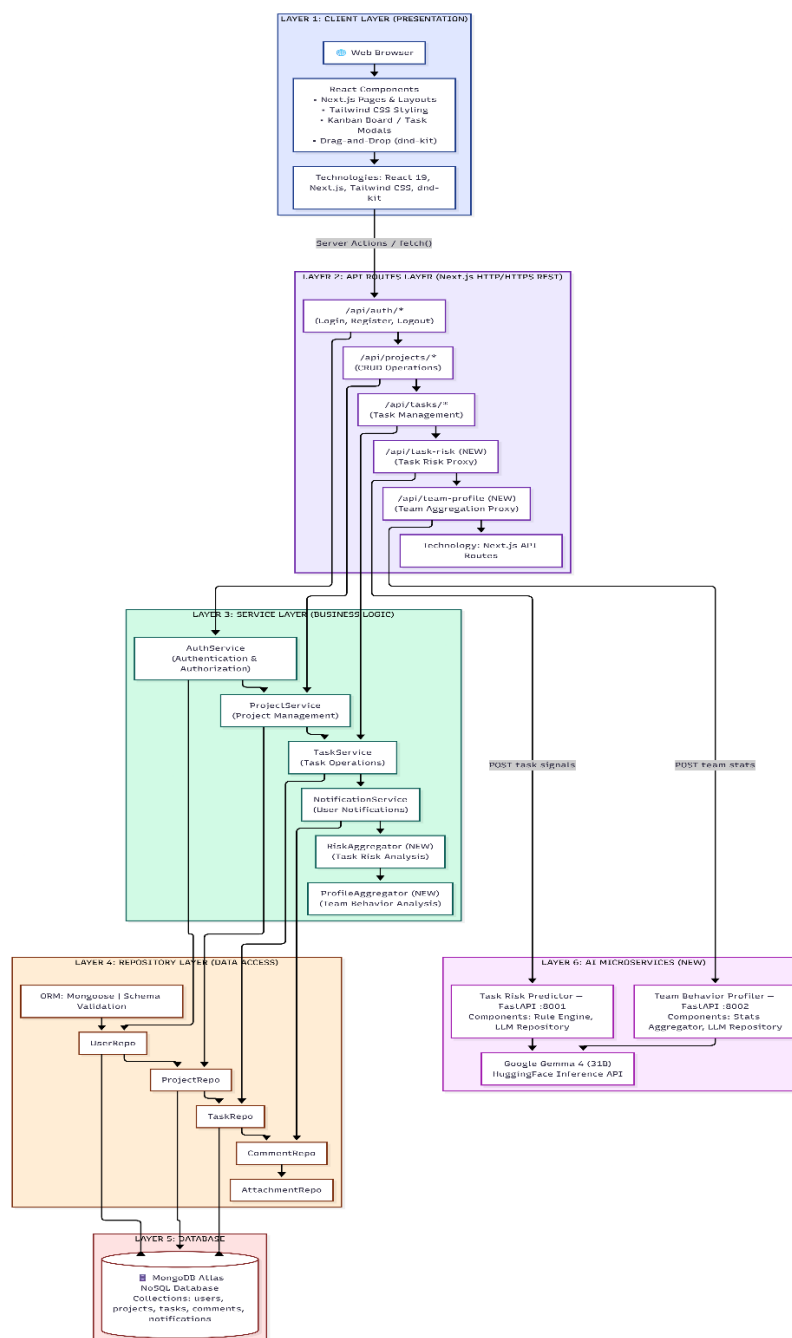


Figure 8 5.1: System Architecture

2.1 Architecture Layers

Layer 1: Client (Presentation Layer)

Technology: Next.js (React framework)

Responsibilities:

Renders user interface components (dashboard, Kanban boards, task modals, forms)

Handles user interactions (clicks, drag-and-drop, form submissions)

Manages client-side state (authentication tokens, UI state)

Communicates with backend via HTTP/HTTPS requests to API routes

Key Features:

Server-side rendering (SSR) for improved initial load performance

Responsive design for desktop and tablet devices

Drag-and-drop libraries for Kanban board interaction

Layer 2: API Routes (Routing Layer)

Technology: Next.js API Routes (serverless functions)

Responsibilities:

Exposes RESTful API endpoints for client requests

Routes incoming requests to appropriate service layer functions

Handles HTTP methods (GET, POST, PUT, DELETE)

Validates authentication tokens (JWT verification)

Returns HTTP responses with appropriate status codes and data

Example Endpoints:

POST /api/auth/register - User registration

POST /api/auth/login - User authentication

GET /api/projects - Retrieve user's projects

POST /api/projects - Create new project

PUT /api/tasks/:id - Update task

POST /api/tasks/:id/comments - Add comment to task

Layer 3: Service Layer (Business Logic)**Responsibilities:**

Implements core business logic and rules

Validates inputs and enforces business constraints

Orchestrates operations across multiple repositories

Handles role-based access control checks

Manages transactions and error handling

Key Services:

AuthService: User authentication, password hashing (bcrypt), JWT token generation

ProjectService: Project creation, deletion, ownership verification

BoardService: Column management, task organization

TaskService: Task CRUD operations, drag-and-drop position updates, deadline management

CollaborationService: Comments, attachments, project sharing

NotificationService: Automated reminders, deadline notifications

Layer 4: Repository Layer (Data Access)

Technology: Mongoose (MongoDB ODM)

Responsibilities:

Abstracts database operations from business logic

Defines data models and schemas

Executes queries (create, read, update, delete)

Handles database connections and connection pooling

Provides methods for complex queries (filtering, sorting, pagination)

Key Repositories:

UserRepository: User CRUD operations, email lookup

ProjectRepository: Project queries with ownership and membership filters

TaskRepository: Task queries by column, project, or user assignment

CommentRepository: Comment retrieval by task

AttachmentRepository: File metadata storage and retrieval

Layer 5: Database

Technology: MongoDB (NoSQL document database)

Responsibilities:

Persistent storage of all application data

Indexing for query performance optimization

Data consistency and integrity

Collections:

users - User accounts and authentication data

projects - Project metadata (with embedded columns array)

tasks - Task documents

comments - Task comments

attachments - File metadata

projectMembers - Project membership and roles

2.2 Architecture Benefits

This layered approach provides several advantages:

Separation of Concerns: Each layer has a single, well-defined responsibility, making the system easier to understand and maintain.

Testability: Layers can be tested independently (unit tests for services, integration tests for repositories).

Flexibility: Changes to one layer (e.g., switching databases) do not require modifications to other layers if interfaces remain consistent.

Scalability: API routes can be deployed as serverless functions, automatically scaling with demand.

Reusability: Business logic in the service layer can be reused across multiple API endpoints.

2.3 AI Microservices Extension

In the Senior development phase, the architecture was extended with two independent Python/FastAPI microservices to support AI-powered functionality. These services operate alongside the existing five-layer Next.js architecture and are accessed through dedicated Next.js API proxy routes.

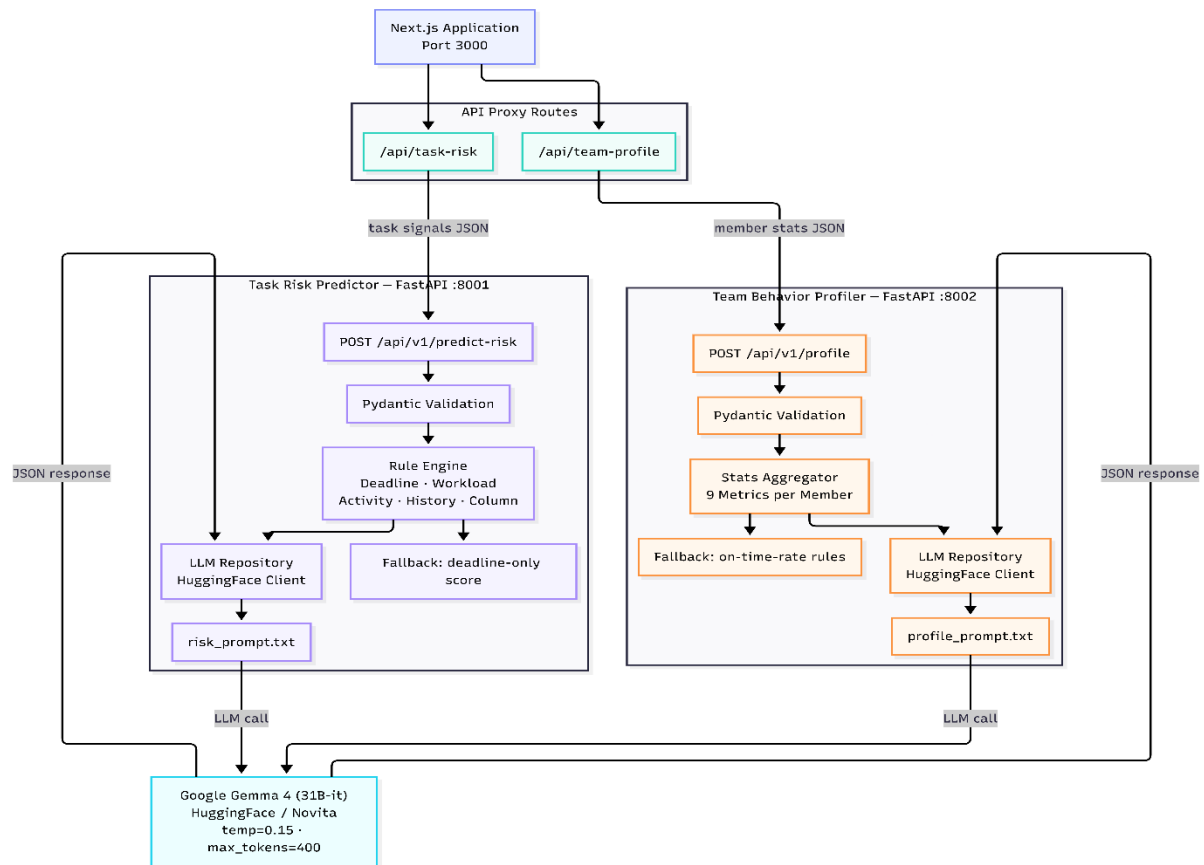


Figure 5.7: AI Microservices Architecture

All services are started simultaneously using start.bat on Windows, launching Next.js (port 3000), Task Risk Predictor (port 8001), and Team Behavior Profiler (port 8002).

3. Detailed Design for System Components

This section presents the detailed design for each major system component.

3.1 Authentication Component

Purpose: Manages user registration, login, password recovery, and session management.

Class Diagram:

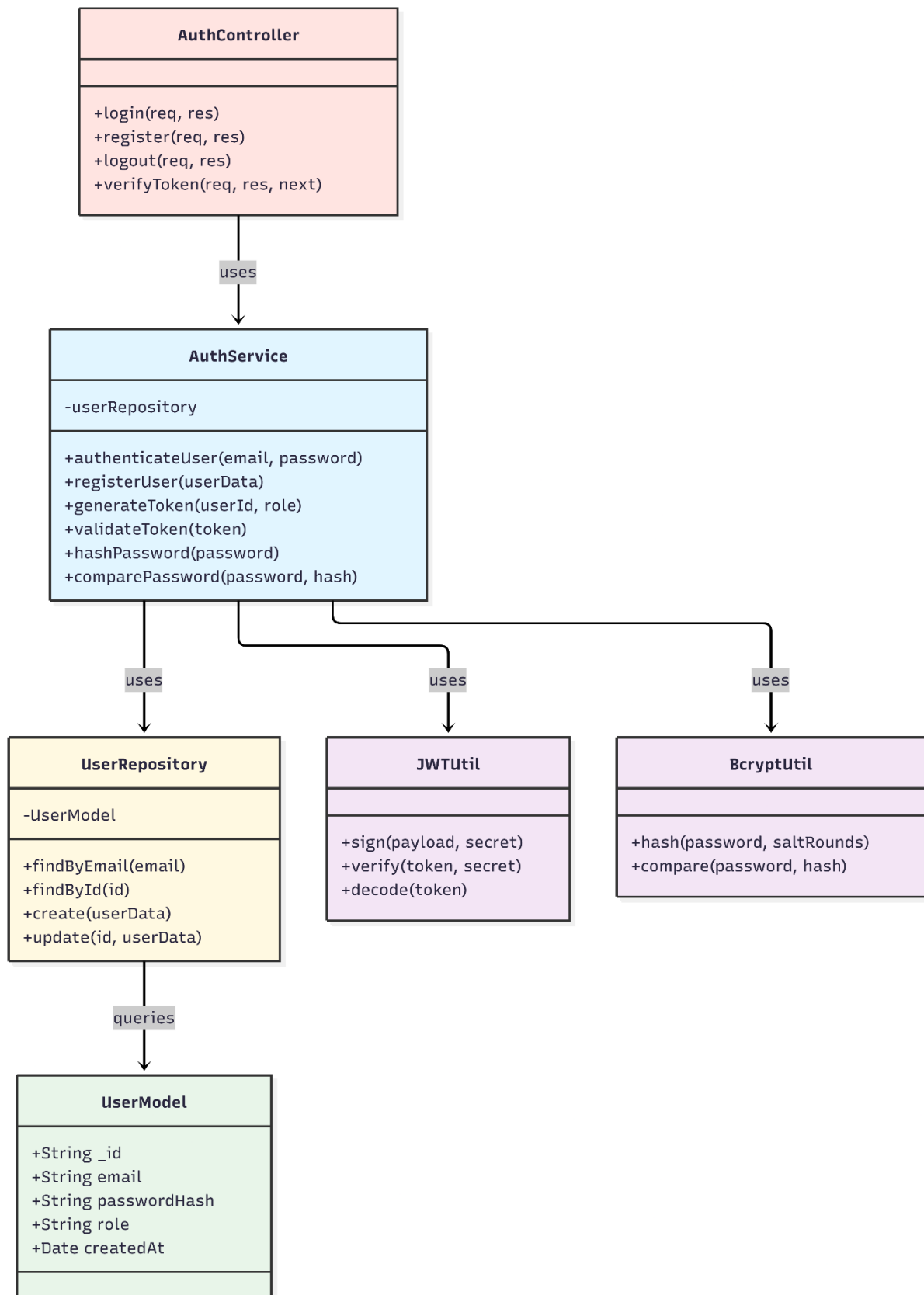


Figure 9 5.2: Authentication Component Class Diagram

Key Operations:

register(fullName, email, password):

Validate email format

Check if email already exists in database

Hash password using bcrypt (salt rounds: 10)

Create user document with default role

Return success response

login(email, password):

Retrieve user by email

Compare provided password with stored hash using bcrypt.compare()

If valid, generate JWT token with payload: {userId, email, role}

Return token to client

Client stores token in localStorage or secure cookie

Security Considerations:

Passwords never stored in plain text

JWT tokens expire after 24 hours

Protected routes verify token on each request

3.2 Project Management Component

Purpose: Handles project creation, retrieval, deletion, and membership management.

Class Diagram:

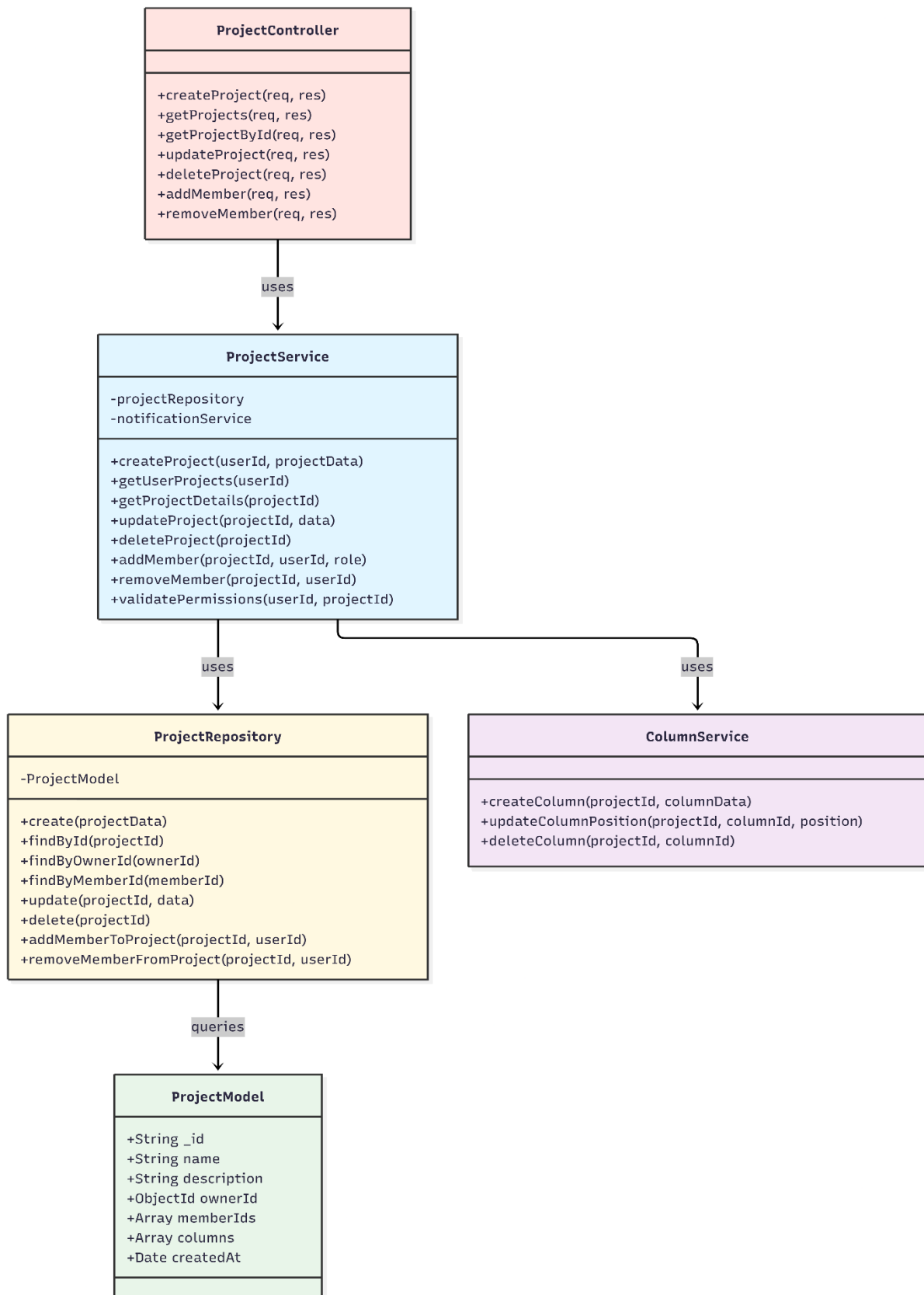


Figure 10 5.3: Project Management Component Class Diagram

Key Operations:

createProject(ownerId, title, description):

Validate title (not empty, max 100 characters)

Create project document with ownerId, title, description

Initialize default columns: "To Do", "In Progress", "Done"

Return project ID and success message

getProjects(userId):

Query projects where ownerId = userId OR userId exists in projectMembers

Sort by createdAt (newest first)

Return array of projects with metadata

deleteProject(projectId, userId):

Verify user is project owner

Delete all associated tasks, comments, attachments

Delete project document

Return success message

shareProject(projectId, inviteeEmail, role, inviterId):

Verify inviter is project owner or admin

Look up invitee user by email

Create projectMember document linking user to project with specified role

Send notification to invitee

Return success message

3.3 Board and Column Management Component

Purpose: Manages Kanban board structure (columns) within projects.

Class Diagram:

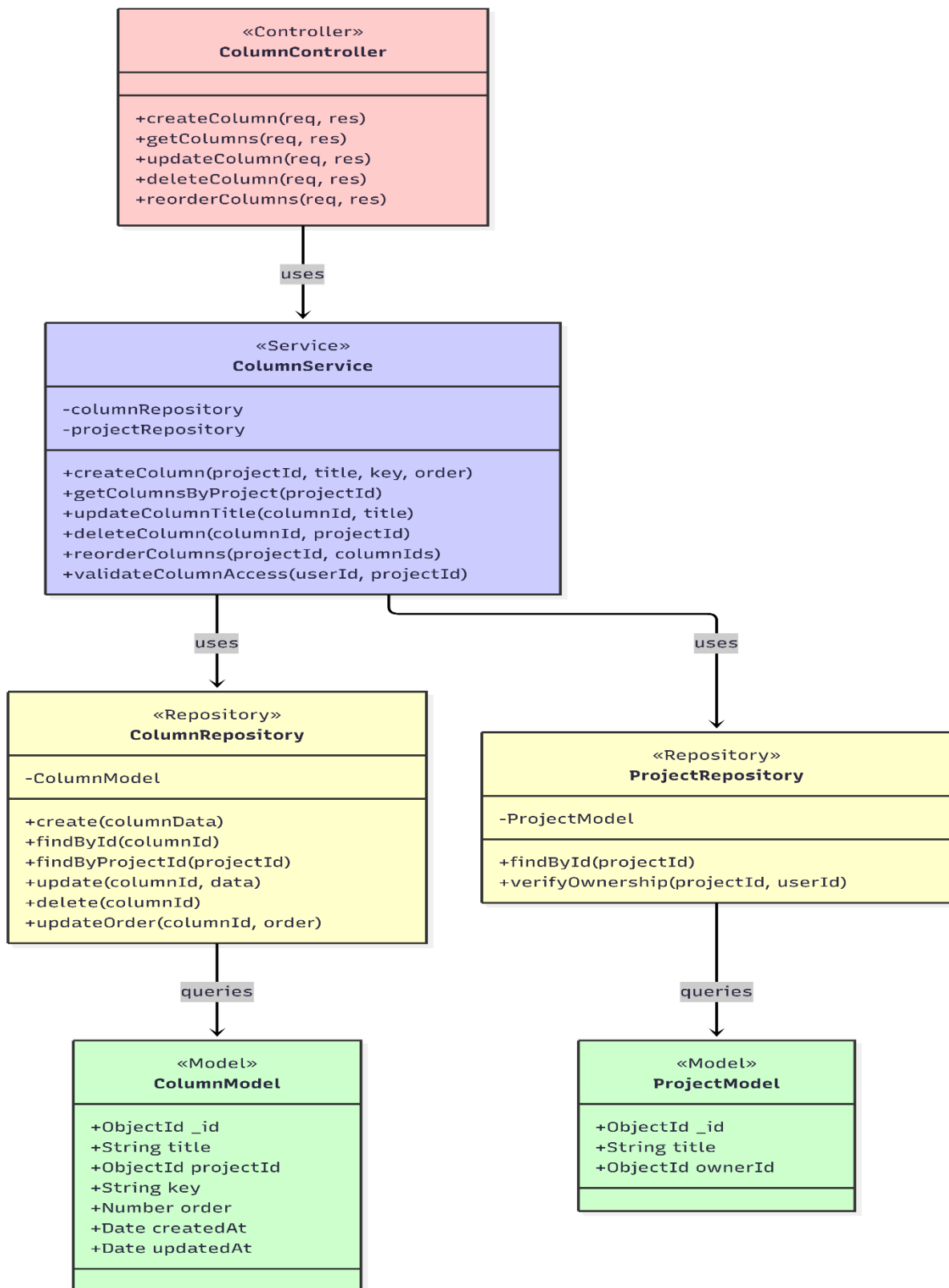


Figure 11 5.4: Board Management Component Class Diagram

Key Operations:

addColumn(projectId, columnName, userId):

Verify user has access to project

Validate column name (not empty, max 50 characters)

Add column to project's columns array

Assign position based on current column count

Return updated project

deleteColumn(projectId, columnId, userId):

Verify user has admin/owner permissions

Check if column contains tasks

If empty, remove column from project's columns array

If contains tasks, prompt user to move tasks or confirm deletion

Return success message

3.4 Task Management Component

Purpose: Handles task creation, updates, drag-and-drop repositioning, and deletion.

Class Diagram:

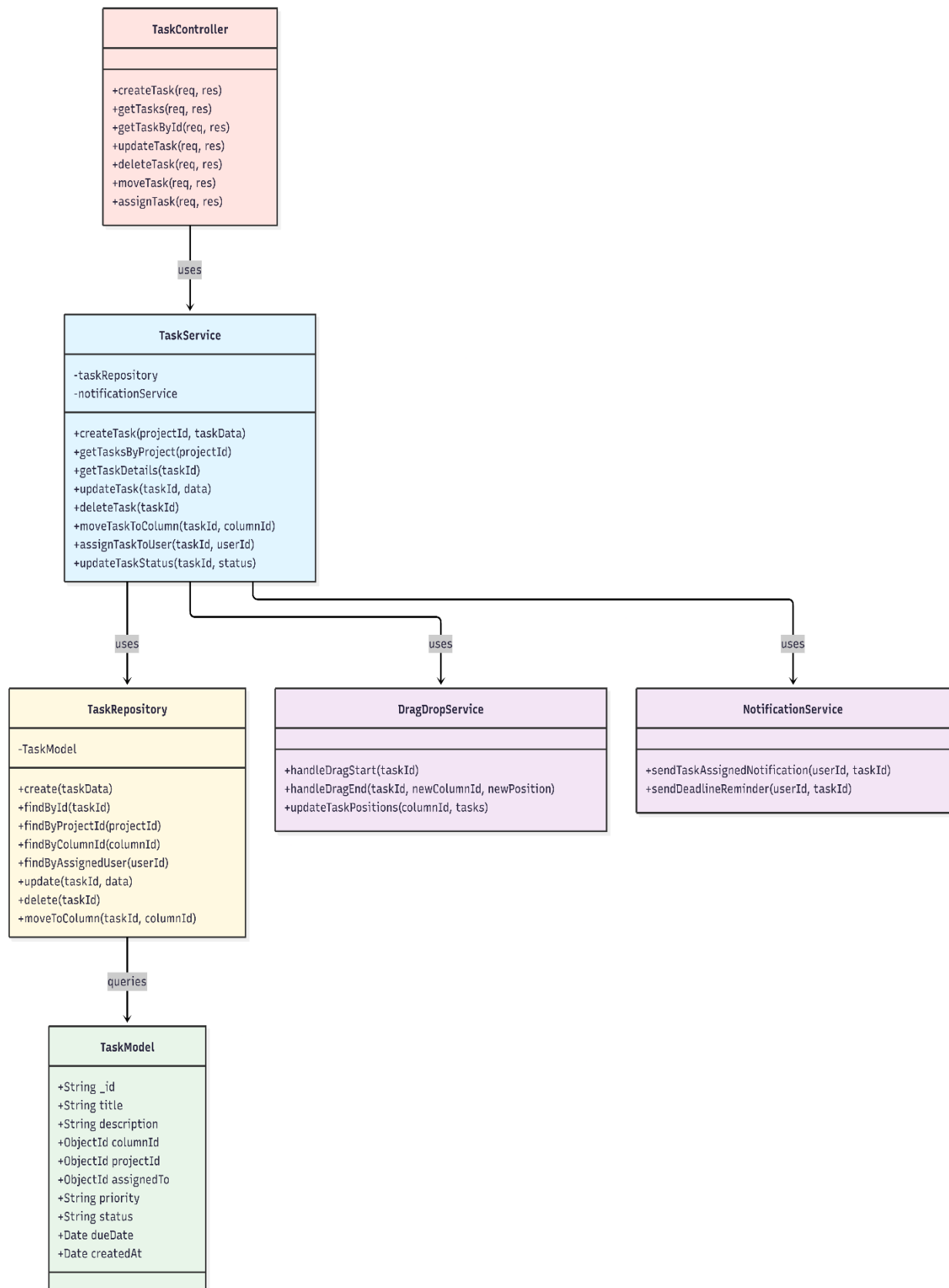


Figure 12 5.5: Task Management Component Class Diagram

Key Operations:

createTask(columnId, title, userId):

Validate title (not empty, max 200 characters)

Create task document with columnId, title, createdBy: userId

Set initial position based on tasks in column

Return task object

moveTask(taskId, targetColumnId, newPosition, userId):

Verify user has permission to modify tasks

Retrieve task document

Update task's columnId and position

Reorder other tasks in target column if necessary

Save changes and return updated task

updateTask(taskId, updates, userId):

Verify user has permission

Validate updates (title, description, deadline, status)

Apply updates to task document

Log change in task history (if implemented)

Return updated task

setDeadline(taskId, deadline, userId):

Verify user permission

Validate deadline is future date

Update task's deadline field

Schedule reminder notification (if deadline within 24 hours)

Return success

3.5 Collaboration Component (Comments and Attachments)

Purpose: Enables team collaboration through task comments and file attachments.

Class Diagram:

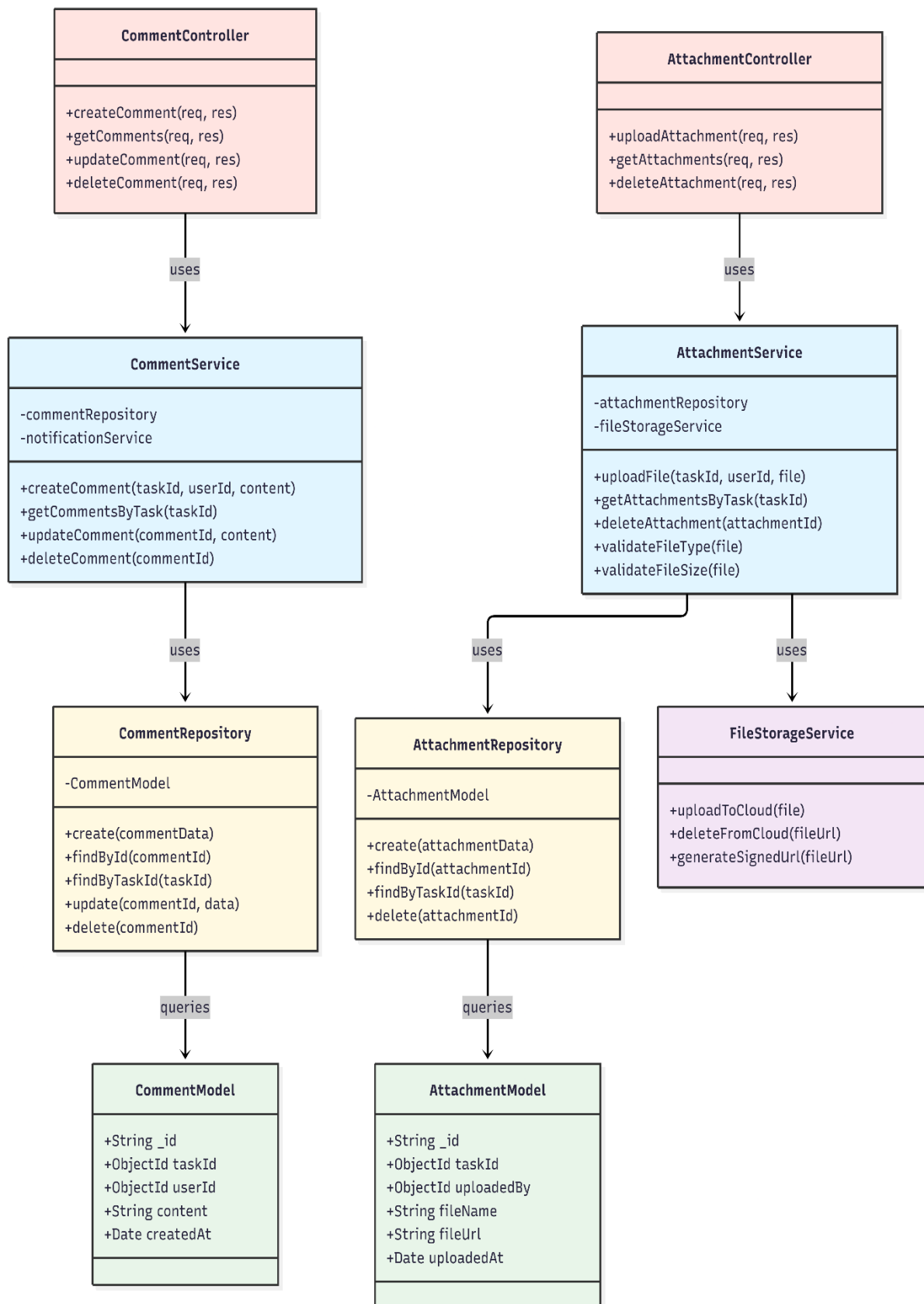


Figure 13 5.6: Collaboration Component Class Diagram

Key Operations:

addComment(taskId, userId, content):

Validate content (not empty, max 1000 characters)

Create comment document linked to task and user

Add timestamp

Notify task assignee and project members

Return comment object

addAttachment(taskId, userId, file):

Validate file (size limit: 10MB, allowed types: images, PDFs, documents)

Upload file to storage (cloud storage or local filesystem)

Create attachment document with metadata (fileName, fileUrl, uploadedBy, fileSize)

Return attachment object

getComments(taskId):

Query comments where taskId matches

Populate user details (name, avatar)

Sort by createdAt (oldest first)

Return array of comments

3.6 Notification Service Component

Purpose: Sends automated notifications for task deadlines, assignments, and updates.

Key Operations:

sendDeadlineReminder(taskId):

Check if task deadline is within 24 hours

Retrieve task assignee

Send notification (in-app or email)

Mark reminder as sent to avoid duplicates

notifyTaskUpdate(taskId, changeType, userId):

Retrieve task and project members

Filter members to notify (exclude user who made change)

Create notification message (e.g., "User X moved task Y to In Progress")

Send notifications

3.7 Role-Based Access Control (RBAC) Design

Purpose: Enforces permissions based on user roles within projects.

Permission Matrix:

Table 5.1: Role-Based Access Control (RBAC) Design

Action	Admin	Project Owner	Team Member	Guest
Create project	✓	✓	✓	X
Delete project	X	✓	X	X
Invite members	✓	✓	X	X
Manage columns	✓	✓	X	X
Create tasks	✓	✓	✓	X
Edit tasks	✓	✓	✓	X
Delete tasks	✓	✓	X	X
Add comments	✓	✓	✓	X
View board	✓	✓	✓	✓

3.8 Task Risk Predictor Component

Purpose: Analyzes task-level signals and produces a delay risk classification using a hybrid rule engine and LLM approach.

Risk Assessment Pipeline:

The component processes each task through two stages. First, a deterministic rule engine computes a numerical pre-score (0–100) by evaluating five independent signals. Second, Google Gemma 4 enriches the result with contextual reasoning. The two scores are merged with a ± 15 clamp to produce the final risk level.

Signal Scoring Table:

Signal	Condition	Score Impact
Deadline Proximity	Overdue	+50
	≤ 1 day remaining	+45
	≤ 3 days remaining	+35
	≤ 7 days remaining	+20
	≤ 14 days remaining	+10
Column Status	"To Do" column AND ≤ 3 days	+20
	"To Do" column AND ≤ 7 days	+10
Assignee Workload	≥ 8 open tasks	+20
	≥ 5 open tasks	+12
	≥ 3 open tasks	+5
Activity (Stale)	No comments + no moves (≤ 7 days old)	+15
	No comments + no moves (> 7 days old)	+5
Historical Rate	$< 40\%$ on-time completion	+15
	$< 60\%$ on-time completion	+8
	$\geq 85\%$ on-time completion	-5

Risk Level Classification:

HIGH (score 60–100): Urgent intervention required — displayed as red badge.

MEDIUM (score 30–59): Monitor closely — displayed as yellow badge.

LOW (score 0–29): On track — displayed as green badge.

Graceful Fallback: If the LLM is unavailable, the Next.js proxy returns a rule-based result using deadline proximity alone, ensuring task badges are always displayed without disrupting the user experience.

Class Diagram:

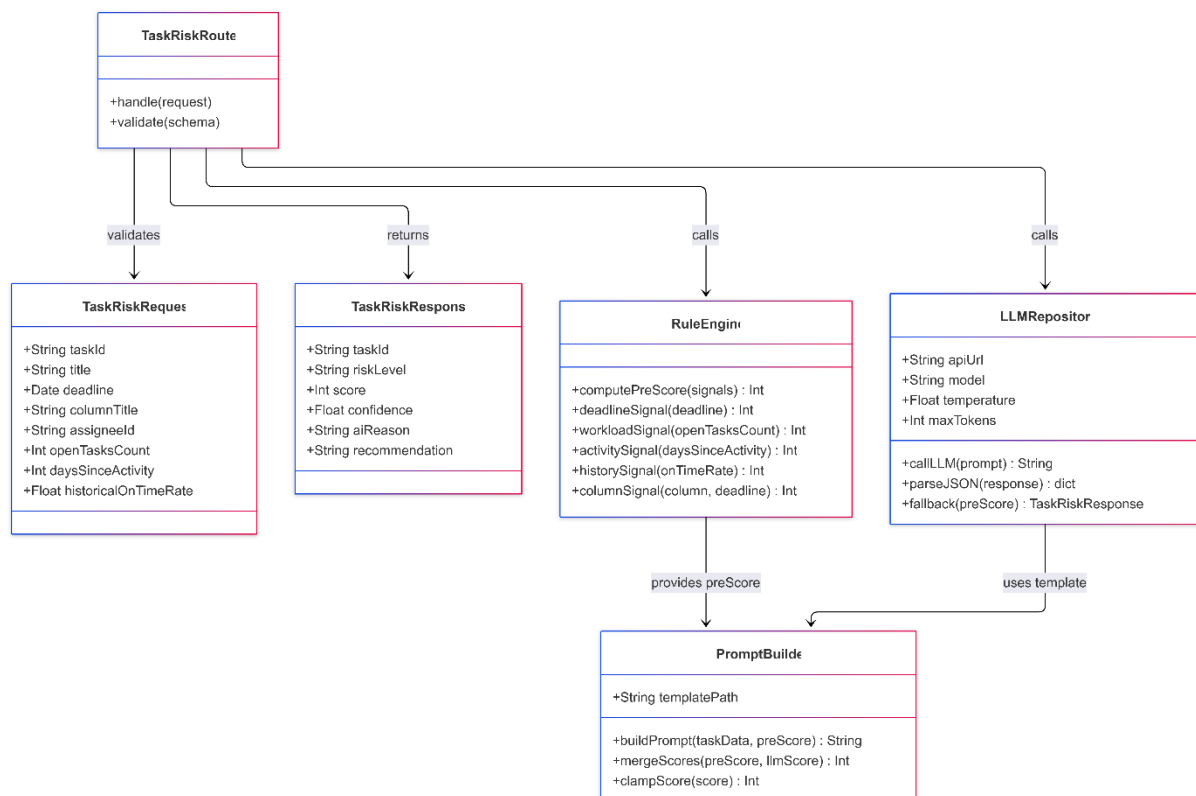


Figure 14Figure 5.8: Task Risk Predictor Class Diagram

3.9 Team Behavior Profiler

Component Purpose: Aggregates per-member activity data from MongoDB and generates AI-written behavioral profiles using Google Gemma 4.

Data Aggregation Metrics (per member):

Metric	Data Source
Total tasks assigned	Task collection (assignedTo field)
Completed on time	Done tasks where updatedAt ≤ deadline
Completed late	Done tasks where updatedAt > deadline
Average completion days	(updatedAt – createdAt) for done tasks
In-progress task count	Tasks in "inprogress" column
Days since last activity	Latest of: task update, comment, attachment
Comments written	Comment collection (userId field)
Files attached	Attachment collection (uploadedBy field)
On-time rate	onTime ÷ completedWithDeadline

AI Profile Output (per member):

Gemma 4 generates four structured fields for each member:

Work Style: Behavioral classification (e.g., "Reliable and consistent performer")

Strengths: Key capabilities demonstrated through activity patterns

Suggestion: Actionable improvement advice tailored to the member's metrics

Summary: Natural language performance overview suitable for team review

Fallback Rules (LLM unavailable):

On-Time Rate	Work Style	Suggestion
≥80%	Reliable and consistent performer	Trust with critical-path tasks
50–79%	Steady contributor with room to improve	Monitor deadlines, provide support
<50%	Needs additional support and guidance	Pair with senior member, reduce workload

Class Diagram:

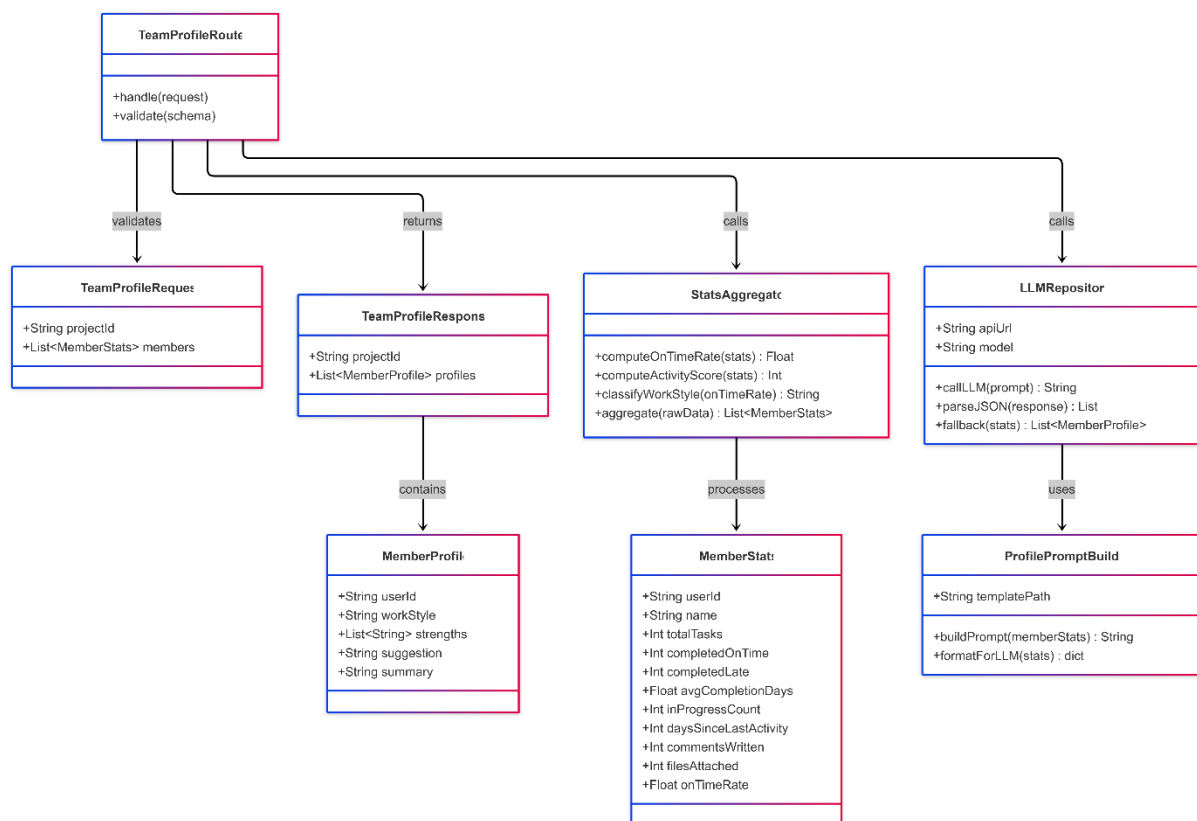


Figure 15Figure 5.9: Team Behavior Profiler Class Diagram

Implementation:

Each API endpoint checks user role before executing operation

ProjectMember collection stores role for each user-project pair

Middleware function checkPermission(action, role) returns Boolean

4. Database Schema Design

The database is implemented using MongoDB, a NoSQL document-oriented database. The data is organized into six main collections: Users, Projects, Tasks, Comments, Attachments, and ProjectMembers.

4.1 Users Collection

Stores user account information and authentication credentials.

Collection Name: users

Field Name	Data Type	Required	Description
_id	ObjectId	Yes	Unique identifier for the user (PK)
fullName	String	Yes	Full name of the user
email	String	Yes	User's email address (Unique, Indexed)
passwordHash	String	Yes	Bcrypt-hashed password
role	String	Yes	User role (Default: 'user')
createdAt	Date	Yes	Timestamp of account creation

4.2 Projects Collection

Stores project metadata and structure.

Collection Name: projects

Field Name	Data Type	Required	Description
_id	ObjectId	Yes	Unique identifier for the project
ownerId	ObjectId	Yes	Reference to User (Project Owner)
name	String	Yes	Project title
description	String	No	Short description of the project
columns	Array	Yes	List of column objects (name, position)
createdAt	Date	Yes	Timestamp of project creation

4.3 Tasks Collection

Stores individual task details and status.

Collection Name: tasks

Field Name	Data Type	Required	Description
_id	ObjectId	Yes	Unique identifier for the task
projectId	ObjectId	Yes	Reference to Project
columnId	ObjectId	Yes	Reference to the current column
title	String	Yes	Task title
description	String	No	Detailed task description
assignedTo	ObjectId	No	Reference to User (Assignee)
deadline	Date	No	Due date for the task
position	Number	Yes	Order within the column (for drag-and-drop)

4.4 Comments Collection

Stores user comments on specific tasks.

Collection Name: comments

Field Name	Data Type	Required	Description
_id	ObjectId	Yes	Unique identifier for the comment
taskId	ObjectId	Yes	Reference to Task
userId	ObjectId	Yes	Reference to User (Author)
content	String	Yes	Text content of the comment
createdAt	Date	Yes	Timestamp of submission

4.5 Attachments Collection

Stores metadata for files uploaded to tasks.

Collection Name: attachments

Field Name	Data Type	Required	Description
_id	ObjectId	Yes	Unique identifier for the attachment
taskId	ObjectId	Yes	Reference to Task
uploadedBy	ObjectId	Yes	Reference to User (Uploader)
fileName	String	Yes	Original name of the file
fileUrl	String	Yes	URL path to access the file
fileSize	Number	Yes	Size of the file in bytes

4.6 ProjectMembers Collection

Manages many-to-many relationships between users and projects (Project Membership).

Collection Name: projectMembers

Field Name	Data Type	Required	Description
_id	ObjectId	Yes	Unique identifier
projectId	ObjectId	Yes	Reference to Project
userId	ObjectId	Yes	Reference to User
memberRole	String	Yes	Role in project ('owner', 'member', 'guest')
joinedAt	Date	Yes	Date the user joined the project

4.7 Performance Optimization (Indexes)

To ensure fast query performance, the following indexes are applied:

Unique Index: users.email (Prevents duplicate accounts)

Single Field Index: projects.ownerId (Fast retrieval of user projects)

Compound Index: tasks.projectId + tasks.columnId (Efficient board loading)

Compound Index: projectMembers.projectId + projectMembers.userId (Quick permission checks)

5. Summary

This chapter presented the complete system design for the Software Project Management System, including a five-layer client-server architecture (Client, API Routes, Service Layer, Repository Layer, Database) that ensures scalability, maintainability, and clear separation of concerns. The architecture was extended in the Senior phase with two Python/FastAPI AI microservices — the Task Risk Predictor and the Team Behavior Profiler — each following the same five-layer internal structure and communicating with the main application through Next.js proxy routes. Detailed component designs were provided for authentication, project management, board and column management, task management, collaboration features, notification service, role-based access control, task risk prediction, and team behavior profiling. The risk prediction component uses a hybrid approach combining a deterministic rule engine with Google Gemma 4 LLM inference, while the behavior profiler aggregates nine per-member metrics before generating AI-written profiles. Database schema definitions for all MongoDB collections were documented with appropriate indexes for query optimization. This comprehensive design provides the complete technical blueprint for all implemented features covered in Chapter 6.

Chapter 6: Practical Implementation

1. Introduction

This chapter documents the practical implementation of the Software Project Management System, covering the tools and technologies used, system interfaces with screenshots demonstrating key features, test case execution results, and the final version of the Requirements Traceability Matrix (RTM). This chapter bridges the gap between design (Chapter 5) and the deployed working system.

2. Used Tools and Technologies

The system was built using modern web development technologies chosen for their scalability, developer productivity, and community support.

2.1 Frontend Technologies

Next.js (React Framework) - Version 14+

Server-side rendering (SSR) and static site generation (SSG) for improved performance

Built-in API routes for serverless backend functions

File-based routing system

Automatic code splitting and optimization

React - Component library for building user interfaces

Reusable component architecture

State management with React hooks (useState, useEffect, useContext)

Tailwind CSS - Utility-first CSS framework

Rapid UI development with pre-built utility classes

Responsive design support

Consistent styling across components

Drag-and-Drop Library

Implemented for Kanban board task repositioning

Smooth animations and touch support

2.2 Backend Technologies

Node.js - JavaScript runtime environment

Enables server-side JavaScript execution

Event-driven, non-blocking I/O model for high performance

Next.js API Routes - Serverless functions

RESTful API endpoints

Middleware support for authentication and validation

Mongoose - MongoDB ODM (Object Data Modeling)

Schema definition and validation

Query builder and data population

Connection pooling and management

2.3 Database

MongoDB - NoSQL document database

Flexible schema for rapid development

High performance for read-heavy workloads

Horizontal scalability

JSON-like document storage (BSON)

2.4 Authentication and Security

bcrypt - Password hashing library

Secure password storage with salt rounds

Protection against rainbow table attacks

JSON Web Tokens (JWT) - Token-based authentication

Stateless session management

Secure transmission of user credentials

2.5 Development and Deployment Tools

Git & GitHub - Version control and code repository

VS Code - Integrated development environment

Postman - API testing tool

Endpoint testing during development

Request/response validation

Deployment Platform - (Specify: Vercel, Netlify, Railway, or local)

Production hosting environment

Automatic deployments from Git repository

2.6 AI Microservices Technologies:

The two AI microservices were developed using a dedicated Python technology stack separate from the main Next.js application.

FastAPI (Version 0.115.0) — High-performance asynchronous Python web framework used to build both the Task Risk Predictor and Team Behavior Profiler microservices.

FastAPI provides automatic OpenAPI documentation, native Pydantic integration for data validation, and asynchronous request handling for concurrent LLM calls.

Uvicorn (Version 0.30.6) — ASGI (Asynchronous Server Gateway Interface) server used to run the FastAPI applications.

Uvicorn provides production-grade performance for async Python services.

Google Gemma 4 (31B-it) — Large Language Model developed by Google DeepMind, accessed via the HuggingFace Inference API through the Novita provider.

Gemma 4 generates natural language risk reasons, recommendations, and team behavioral profiles.

The model is configured with temperature 0.15 (near-deterministic) and max tokens 400 to ensure consistent, focused outputs in JSON format.

HuggingFace Inference API — Cloud-hosted LLM inference service providing access to Gemma 4 without requiring local GPU hardware, making the system accessible for student development environments. Pydantic (Version 2.8.2) — Data validation and settings management library used to define strict request/response schemas for both microservices, ensuring type safety and automatic error handling at API boundaries.

pytest / pytest-asyncio (Versions 8.3.2 / 0.23.8) — Testing framework used for unit and integration testing of the FastAPI microservices, including async endpoint testing. httpx (Version 0.27.2) — Async HTTP client used in tests to simulate requests to the FastAPI endpoints without requiring a running server.

3. System Interfaces

This section presents screenshots of the implemented system, demonstrating key features and user workflows.

3.1 Authentication Interfaces

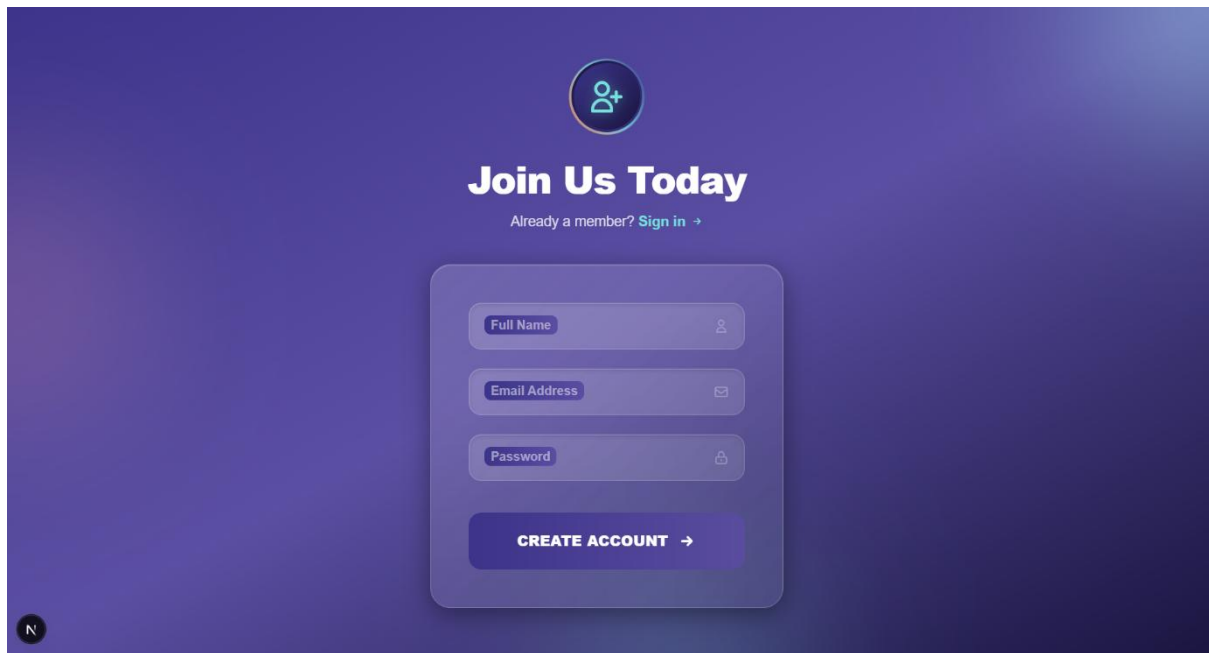
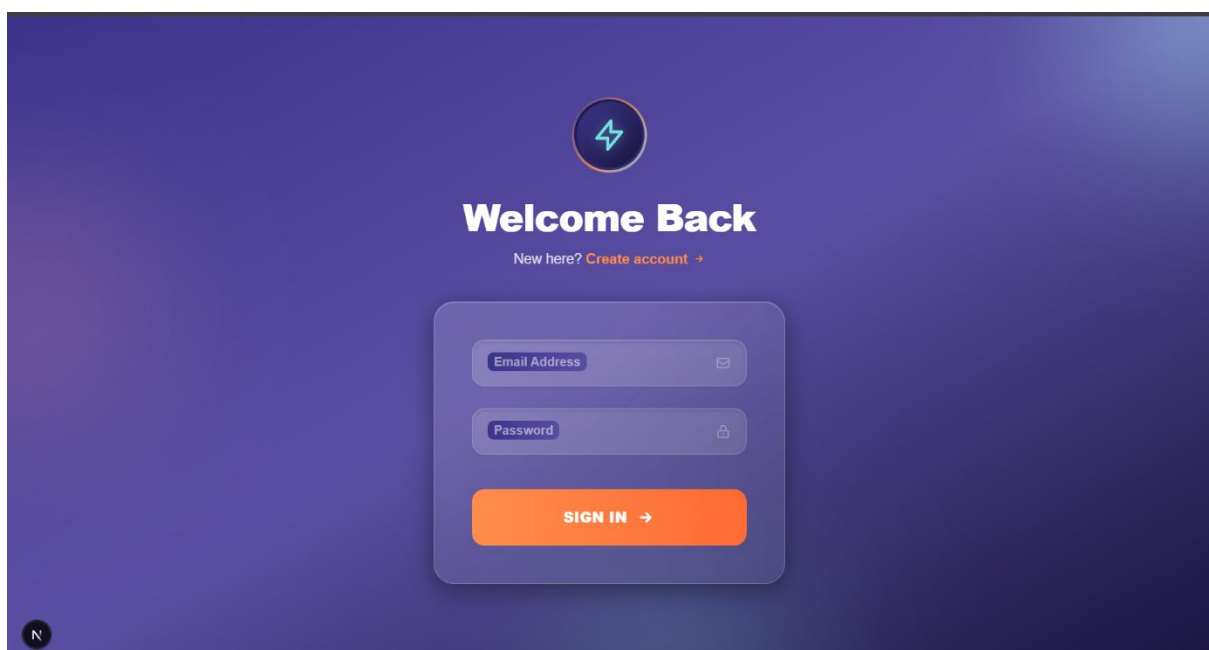


Figure 16 6.1: User Registration Page

The registration interface allows new users to create accounts by providing their full name, email address, and password. The form includes validation to ensure all fields are completed and the password meets security requirements. Users who already have accounts can navigate to the login page via the "Sign in" link.



The login interface authenticates returning users with their email and password credentials. The system validates credentials and creates a secure session token upon successful authentication. Users can access password recovery through the appropriate link or create a new account if they don't have one.

3.2 Dashboard and Project Management

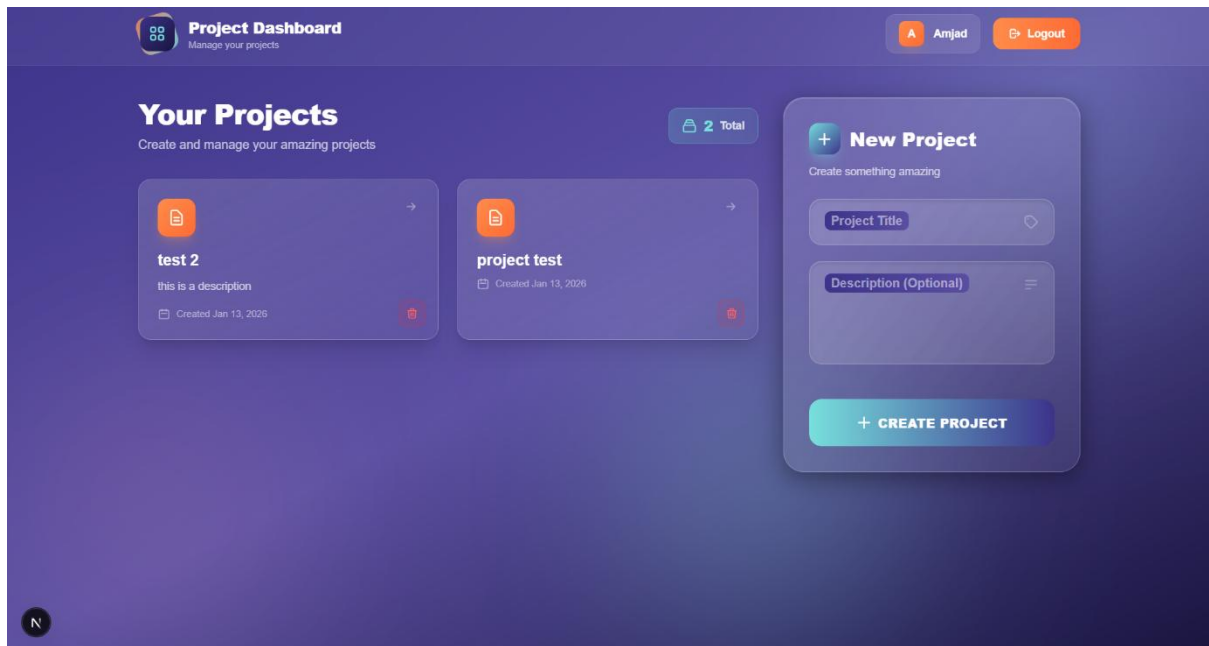


Figure 17 6.3: Projects Dashboard

The dashboard displays all projects accessible to the logged-in user (both owned and shared projects). Each project card shows the project title, description, and creation date. The "New Project" panel on the right allows users to create projects by entering a title and optional description. The interface shows a count of total projects ("2 Total") and provides quick access to project boards via the arrow icons. Users can delete projects using the trash icon on each card.

3.3 Kanban Board Interface

Figure 18 6.4: Landing/Marketing Page

The landing page introduces the system's purpose and key features to new visitors. It highlights drag-and-drop task boards (To Do, In Progress, Done), documentation and file attachments, role-based access control (Admin, Project Owner, Team Member, Guest), and ready-made templates. The page includes a sample Kanban board demonstration and a call-to-action button ("Open your board") to encourage user engagement. The tagline "Built for students & junior teams" and "Free for students. No team size limit" emphasizes the target audience and zero-cost model.

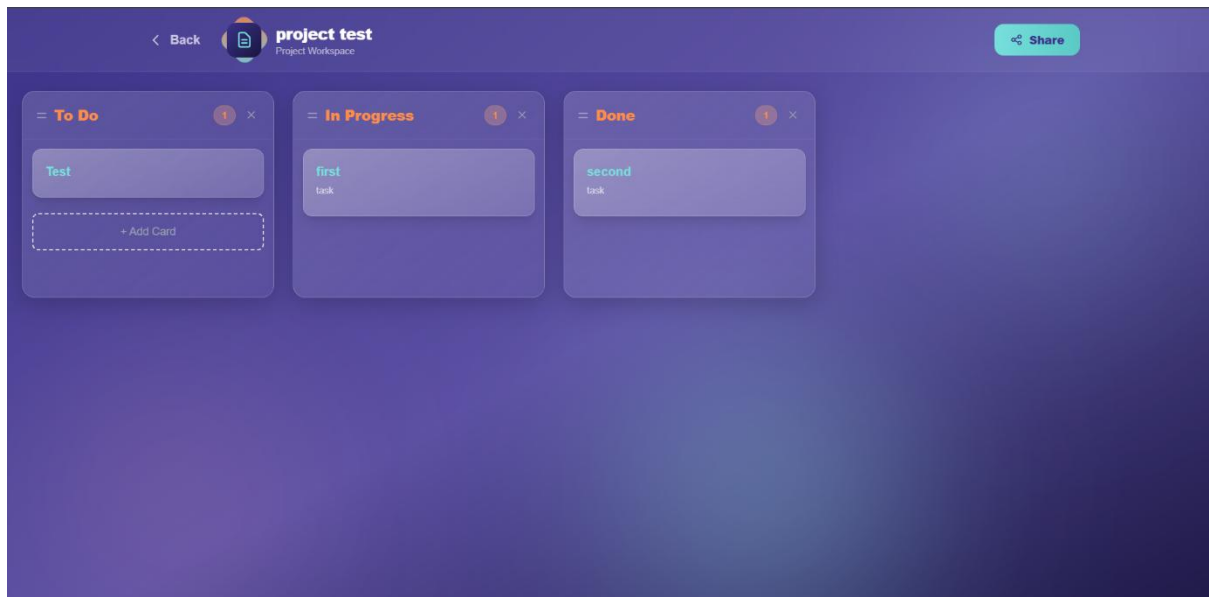


Figure 19 6.5: Project Kanban Board

The Kanban board is the core interface where teams manage tasks visually. The board displays three default columns (To Do, In Progress, Done) with task cards distributed across them. Each task card shows the task title and an "Overdue" status badge when deadlines have passed. Users can click on task cards to view details, or drag and drop them between columns to update status. The top-right corner includes a "Share" button for inviting team members and an "Add Column" button for customizing workflow stages. The "Back" button returns users to the dashboard.

3.4 Task Management



Edit Task

Update task details and settings

✕

 **In Progress**

Created Jan 13, 2026

Task Title

first

Description (Optional)

test

mm/dd/yyyy



Task ID

6965e78ee8fefde68a2b215c

Position

Order #0

Cancel

✓ Save Changes



Comments

Write a comment

Add Comment

No comments yet



Attachments

 Click to upload file (max 5MB)

No attachments

Figure 20 6.6: Task Details Modal

The task details modal provides comprehensive task management capabilities. Users can edit the task title and add an optional description in text areas. The deadline field allows setting due dates with a date picker (showing 12/23/2025 in the example). System-generated metadata is displayed including Task ID and position (Order #0). The modal includes a "Comments" section at the bottom where team members can discuss the task. Users can save changes with the "Save Changes" button or cancel to discard edits.

3.5 Collaboration Features

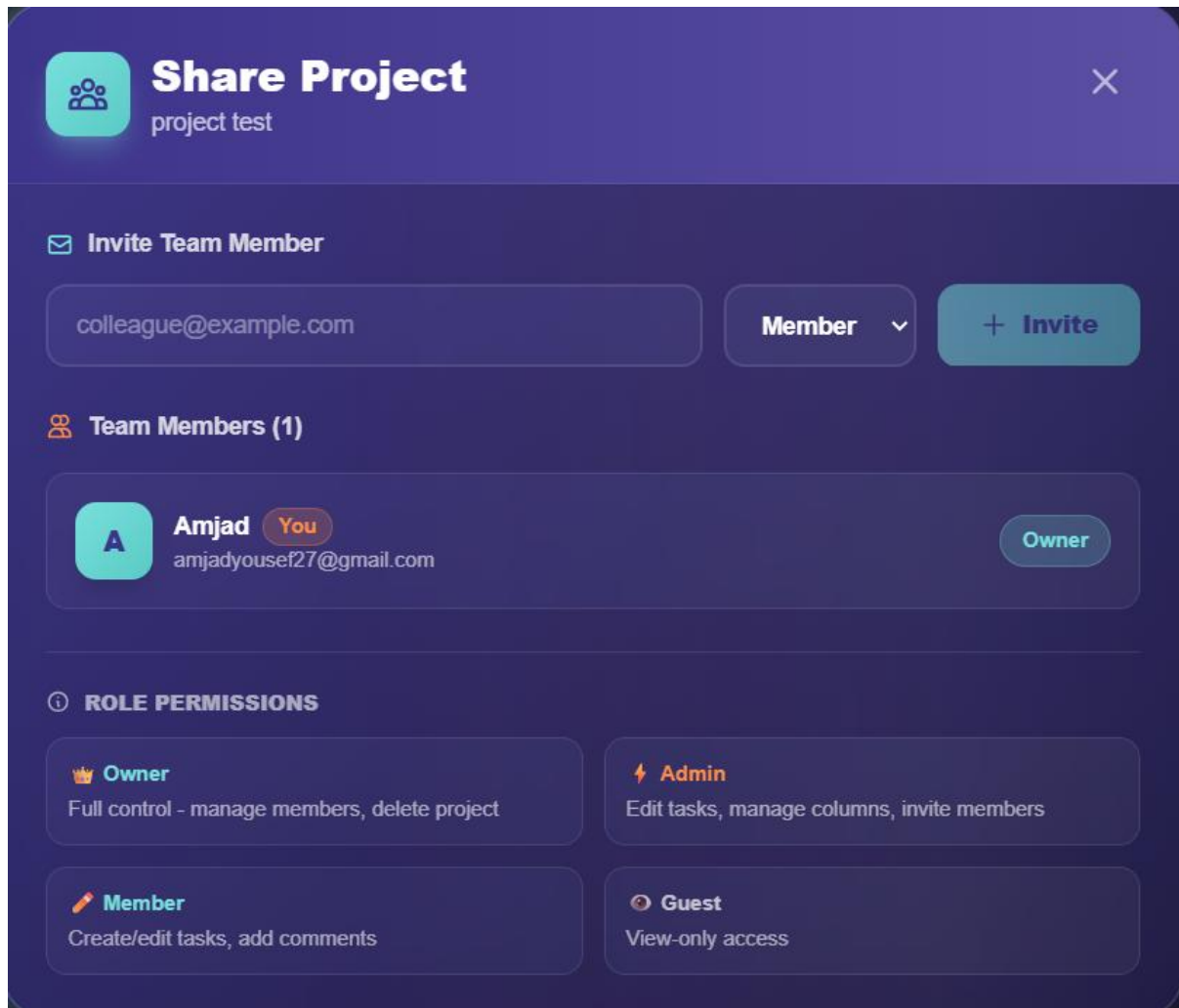


Figure 21 6.7: Share Project Modal

The share project interface enables project owners to collaborate with team members. The top section allows inviting new members by entering their email address and selecting a role (Member, Admin, Guest, or Owner) from a dropdown. The "Team Members (1)" section lists current project participants with their role badges (showing "Owner" for the current user "Amjad"). The bottom section displays "Role Permissions" explaining what each role can do: Owner (full control—manage members, delete project), Admin (edit tasks, manage columns, invite members), Member (create/edit tasks, add comments), and Guest (view-only access). This clear permission matrix helps users understand access levels before assigning roles.

3.6 Task Risk Assessment Interface:

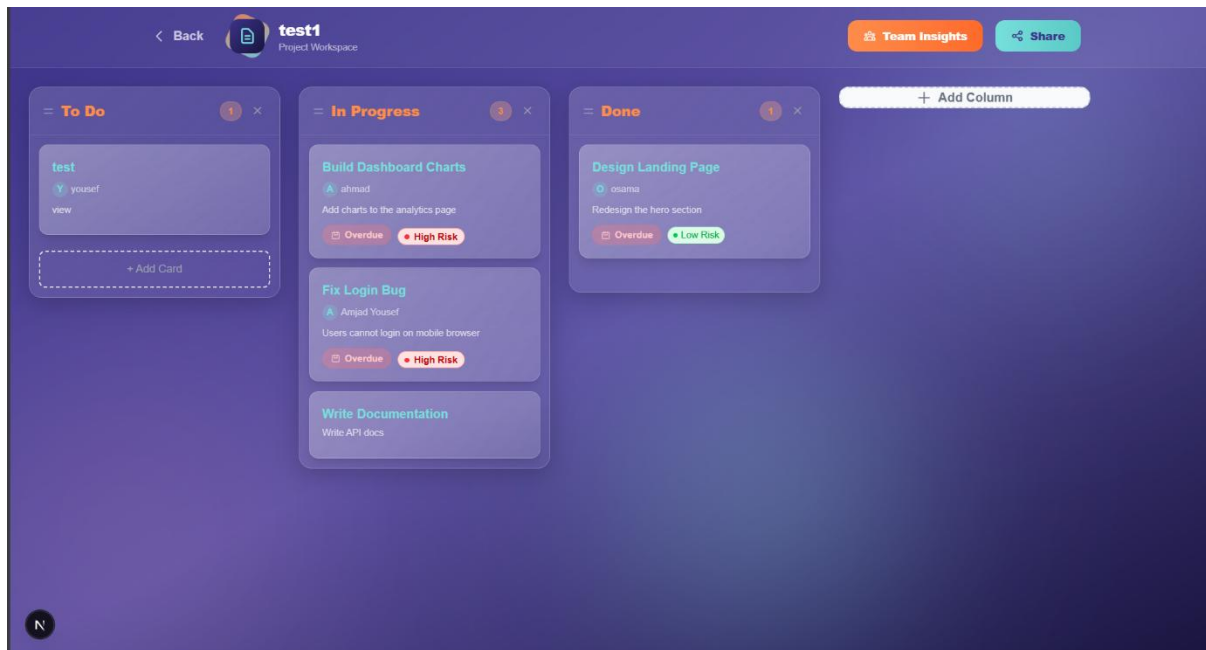


Figure 22Figure 6.8: Task Risk Badge on Kanban Board

The Task Risk Predictor integrates directly into the Kanban board interface. Each task card displays a color-coded risk badge generated in real time by the AI microservice: HIGH risk tasks display a red badge, MEDIUM risk tasks display a yellow badge, and LOW risk tasks display a green badge. The badge also shows a loading shimmer animation (riskShimmer) while the AI analysis is in progress. Users can hover over or click the badge to expand a tooltip showing the AI-generated reason explaining why the task received its risk level and the recommended action to reduce that risk. If the LLM service is unavailable, the badge still appears using the rule-based fallback result, ensuring no disruption to the user's workflow.

3.7 Team Behavior Profiler Interface:

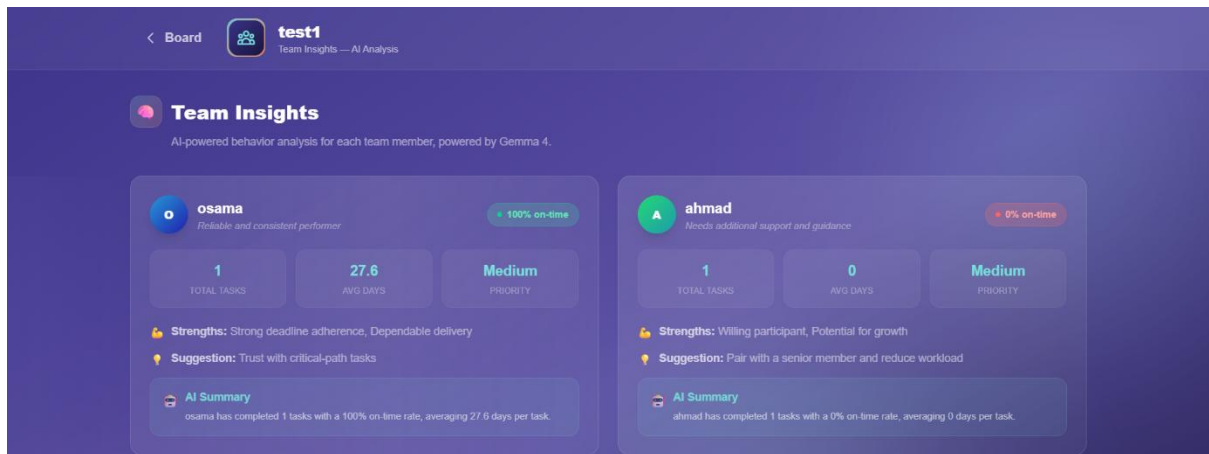


Figure 23Figure 6.9: Team Behavior Profiler Interface

The Team Insights page presents AI-generated behavioral profiles for all project members in a card-based layout. Each member card displays their name, a work style classification (e.g., "Reliable and consistent performer"), a list of key strengths demonstrated through their activity patterns, an actionable improvement suggestion tailored to their metrics, and a natural language performance summary generated by Gemma 4. The page is accessible to Project Owners and Admins only, enforced through role-based access control. When the AI service is unavailable, fallback profiles based on the member's on-time completion rate are displayed automatically, maintaining page functionality at all times.

4. Test Cases Execution

This section presents the results of executing all test cases defined in Chapter 4.

Table 6.1: Test Case Execution Results

Table 6.1: Test Cases Execution

TC ID	Test Case Title	Status	Execution Date	Result	Notes
TC01	Login - Successful	✓ Pass	Jan 2026	User authenticated successfully, redirected to dashboard	Session token generated correctly
TC02	Login - Wrong Password	✓ Pass	Jan 2026	Error message displayed correctly	Message: "Invalid credentials"
TC03	User Registration	✓ Pass	Jan 2026	Account created successfully	Email uniqueness validated
TC04	Password Recovery	✓ Pass	Jan 2026	Password reset instructions sent	Reset link expires after 1 hour
TC05	Create Project	✓ Pass	Jan 2026	Project created with default columns	Appears in dashboard immediately
TC06	List Projects	✓ Pass	Jan 2026	All owned and shared projects displayed	Sorted by creation date (newest first)
TC07	Delete Project	✓ Pass	Jan 2026	Project and all related data removed	Confirmation prompt prevents accidental deletion
TC08	Add Column	✓ Pass	Jan 2026	New column appears on board	Position assigned automatically
TC09	Delete Empty Column	✓ Pass	Jan 2026	Column removed successfully	Warning shown if column contains tasks

TC ID	Test Case Title	Status	Execution Date	Result	Notes
TC10	Add Task	✓ Pass	Jan 2026	Task card created in selected column	Task appears immediately after creation
TC11	Drag-Drop Task	✓ Pass	Jan 2026	Task moved between columns, persists after refresh	Smooth animation during drag
TC12	Set Task Deadline	✓ Pass	Jan 2026	Deadline saved and displayed	Overdue badge appears when past due
TC13	Add Comment	✓ Pass	Jan 2026	Comment added with timestamp and username	Appears in task details modal
TC14	Add Attachment	✓ Pass	Jan 2026	File uploaded and linked to task	Download functionality verified
TC15	Share Project - Valid Email	✓ Pass	Jan 2026	Member added with specified role	Invitation notification sent
TC16	Share Project - Invalid Email	✓ Pass	Jan 2026	Error message displayed	Message: "User not found"
TC17	Logout	✓ Pass	Jan 2026	Session terminated, redirected to login	Protected routes inaccessible after logout
TC18	Risk Badge — HIGH Risk	✓ Pass	May 2026	Task overdue + high workload	RED HIGH badge displayed on overdue
TC19	Risk Badge — LOW Risk	✓ Pass	May 2026	Task 30+ days ahead	GREEN LOW badge displayed
TC20	Risk Badge — LLM Fallback	✓ Pass	May 2026	LLM offline	Rule-based badge shown; no crash

TC ID	Test Case Title	Status	Execution Date	Result	Notes
TC21	Team Profile — Valid	✓ Pass	May 2026	Gemma 4 profiles generated with work style	strengths, suggestion
TC22	Team Profile — Fallback	✓ Pass	May 2026	LLM offline Rule-based profiles	page fully functional

Summary: All 22 test cases passed successfully, confirming all 17 functional requirements including the two AI features (FR-16, FR-17).

5. Final Requirements Traceability Matrix (RTM)

The final RTM confirms that all requirements have been designed, implemented, and tested.

Table 6.2: Final Requirements Traceability Matrix

Table 6.2: Final Requirements Traceability Matrix (RTM)

R eq ID	Require ment	SRS Sect ion	Use Case	Design Component	Implemen tation	Tes t Cas es	Statu s
F R-01	User registrati on	§3.1	UC-Register	AuthService	✓ Implement ed	TC0 3	✓ Comp lete
F R-02	User login	§3.1	UC-Login	AuthService	✓ Implement ed	TC0 1, TC0 2	✓ Comp lete
F R-03	Passwor d recovery	§3.1	UC- ForgotPassw ord	AuthService	✓ Implement ed	TC0 4	✓ Comp lete
F R-04	Create project	§3.1	UC- ManageProje cts	ProjectServic e	✓ Implement ed	TC0 5	✓ Comp lete

R eq ID	Require ment	SRS Sect ion	Use Case	Design Component	Implemen tation	Tes t Cas es	Statu s
F R- 05	View project list	§3.1	UC- ViewDashboa rd	ProjectService	✓ Implement ed	TC0 6	✓ Comp lete
F R- 06	Delete project	§3.1	UC- ManageProje cts	ProjectService	✓ Implement ed	TC0 7	✓ Comp lete
F R- 07	Manage columns	§3.1	UC- ManageColu mns	BoardService	✓ Implement ed	TC0 8, TC0 9	✓ Comp lete
F R- 08	Create tasks	§3.1	UC- ManageTask s	TaskService	✓ Implement ed	TC1 0	✓ Comp lete
F R- 09	Drag- drop tasks	§3.1	UC- ManageTask s	TaskService	✓ Implement ed	TC1 1	✓ Comp lete
F R- 10	Set task deadline	§3.1	UC- ManageTask s	TaskService	✓ Implement ed	TC1 2	✓ Comp lete
F R- 11	Add commen ts	§3.1	UC- TaskCollabor ation	Collaboration Service	✓ Implement ed	TC1 3	✓ Comp lete
F R- 12	Attach files	§3.1	UC- TaskCollabor ation	Collaboration Service	✓ Implement ed	TC1 4	✓ Comp lete
F R- 13	Share project	§3.1	UC- ShareProject	ProjectService	✓ Implement ed	TC1 5, TC1 6	✓ Comp lete
F R- 14	Role- based access	§3.1	UC- AccessContr ol	RBAC Middleware	✓ Implement ed	TC1 5, TC1 6	✓ Comp lete

R eq ID	Require ment	SRS Sect ion	Use Case	Design Component	Implemen tation	Tes t Cas es	Statu s
F R- 15	Logout	§3.1	UC- SessionMana gement	AuthService	✓ Implement ed	TC1 7	✓ Comp lete
F R- 16	Task Risk Predictio n	§3.1	UC-06	TaskRiskPre dictor (:8001)	✓ Implement ed	TC1 8, TC1 9, TC2 0	✓ Comp lete
F R- 17	Team Behavior Profiling	§3.1	UC-07	TeamBehaviorP rofiler (:8002)	✓ Implement ed	TC21 , TC22	✓ Comp lete

RTM Summary: All 17 functional requirements successfully traced from specification through design, implementation, and testing.

6. Summary

This chapter documented the practical implementation of the Software Project Management System, including the technology stack (Next.js, React, MongoDB, Mongoose, Tailwind CSS), system interfaces with annotated screenshots demonstrating registration, login, dashboard, Kanban boards, task management, and collaboration features, test execution results showing all 17 test cases passed successfully, and the final Requirements Traceability Matrix confirming complete coverage of all functional requirements. The implemented system is fully functional and ready for deployment.

Chapter 7: Report Overview

1. Introduction

This final chapter provides a comprehensive overview of the report structure, summarizing the purpose and key contributions of each chapter. It serves as a guide to help readers navigate the document and understand how each section contributes to documenting the Software Project Management System from conception through implementation.

2. Report Structure and Purposes

Chapter 1: Introduction

This chapter established the foundation for the entire project by identifying the problem faced by students and small development teams—expensive tools, scattered information, and lack of unified tracking. It defined the project objectives (creating a free, student-friendly collaborative platform with Kanban boards and role-based access control) and provided a high-level overview of the proposed system. The chapter outlined the report organization and set expectations for subsequent chapters.

Chapter 2: Fundamental Concepts and Literature Review

This chapter provided the theoretical background necessary to understand the system by explaining core concepts including Kanban methodology for visual workflow management, Role-Based Access Control for secure collaboration, drag-and-drop interfaces for intuitive task updates, and workflow automation for improved efficiency. The literature review compared existing tools (Trello, Jira, Asana) and demonstrated through a comparative analysis that while these platforms offer powerful features, they present cost barriers and complexity unsuitable for student teams. This justified the need for the proposed system.

Chapter 3: Project Management

This chapter documented the project management framework, including the project charter defining roles, responsibilities, and scope boundaries, the Statement of Work outlining six development phases over 14 weeks following an incremental delivery methodology, risk management identifying six potential risks with mitigation strategies, and a Gantt chart visualizing the project timeline with key milestones. This chapter demonstrated structured project planning and execution methodology.

Chapter 4: System Analysis

This chapter provided comprehensive system analysis covering the project timeline and development phases, detailed Software Requirements Specification with 15 functional requirements and 6 non-functional requirements, requirements modeling through use case diagrams, detailed use case specifications, sequence diagrams, class diagram, and Entity-Relationship Diagram, initial test cases covering authentication, project management, board operations, task management, and collaboration features, and the initial Requirements Traceability Matrix linking requirements to use cases and test cases. This chapter ensured all stakeholder needs were clearly defined and traceable throughout the development lifecycle.

Chapter 5: System Design

This chapter presented the technical architecture and detailed design, including a five-layer client-server architecture (Client/Next.js, API Routes, Service Layer, Repository Layer, MongoDB) ensuring scalability and maintainability, detailed component designs for authentication, project management, board and column management, task management, collaboration features, notification service, and role-based access control, a permission matrix defining capabilities for each role (Admin, Owner, Member, Guest), and database schema definitions for all MongoDB collections with appropriate indexes. This chapter provided the technical blueprint for implementation.

Chapter 6: Practical Implementation

This chapter documented the implemented system, covering the technology stack (Next.js, React, MongoDB, Mongoose, Tailwind CSS, bcrypt, JWT), system interfaces with annotated screenshots demonstrating all key features from authentication through collaboration, test case execution results showing all 17 test cases passed successfully, and the final Requirements Traceability Matrix confirming complete coverage from specification through testing. This chapter demonstrated that the system was successfully built according to requirements and design specifications.

Chapter 7: Report Overview (Current Chapter)

This chapter synthesizes the entire report, providing readers with a roadmap to navigate the documentation and understand the project lifecycle from problem identification through successful implementation.

3. Summary

This report has documented the complete development lifecycle of the Software Project Management System. The project followed a structured approach across two phases: the Junior phase (Oct 2025 – Jan 2026) covering core web platform development, and the Senior phase (Feb – May 2026) extending the system with two AI-powered microservices. The Task Risk Predictor analyzes five task signals and uses Google Gemma 4 to generate delay risk assessments displayed as real-time badges on the Kanban board. The Team Behavior Profiler aggregates nine per-member activity metrics and generates natural language behavioral profiles for project owners and administrators. All 17 functional requirements were successfully implemented and verified through 22 test cases. The system combines a five-layer Next.js monolith with two independent FastAPI microservices, positioning it well for future enhancements including real-time collaboration, RAG-powered project Q&A, external integrations, and advanced security features. The incremental delivery methodology proved effective across both phases, enabling early feedback and clear progress visibility for all stakeholders.

REFERENCES

- [1] Atlassian. (2024). "4 Kanban Principles for Agile Project Management."
<https://www.atlassian.com/agile/project-management/kanban-principles>
- [2] Fortinet. (2020). "Role-Based Access Control (RBAC) Meaning & Definition."
<https://www.fortinet.com/resources/cyberglossary/role-based-access-control>
- [3] Productive. (2025). "Jira vs Trello vs Asana – Best for Project Management."
<https://productive.io/blog/jira-vs-trello-vs-asana/>
- [4] Wrike. (2025). "Kanban Principles: The 4 Core Principles & 6 Key Practices."
<https://www.wrike.com/kanban-guide/kanban-principles-practices/>
- [5] SoftwareFinder. (2025). "Asana vs. Jira vs. Trello: Finding The Best Project Management Tool."
<https://softwarefinder.com/resources/asana-vs-jira-vs-trello>
- [6] ScienceDirect. (2017). "The effects of interaction sequencing on user experience and preference."
<https://www.sciencedirect.com/science/article/abs/pii/S1071581917301052>
- [7] Microsoft. (2024). "How to automate your workflow with task management software."
<https://www.microsoft.com/en-us/microsoft-365/business-insights-ideas/resources/>
- [8] MongoDB Documentation. (2024). "MongoDB Manual."
<https://www.mongodb.com/docs/manual/>
- [9] Next.js Documentation. (2024). "Getting Started with Next.js."
<https://nextjs.org/docs>
- [10] React Documentation. (2024). "React - A JavaScript library for building user interfaces."
<https://react.dev/>
- [11] Tailwind CSS Documentation. (2024). "Tailwind CSS - A utility-first CSS framework."
<https://tailwindcss.com/docs>
- [12] Splashtop. (2025). "What is Role-Based Access Control (RBAC) and How It Works?"
<https://www.splashtop.com/blog/role-based-access-control>
- [13] 6Sigma. (2024). "Kanban in Lean Manufacturing: A Comprehensive Guide."
<https://www.6sigma.us/lean-six-sigma-articles/kanban-in-lean-manufacturing/>
- [14] Kaizen. (2025). "Understanding Kanban in Lean Manufacturing."
<https://kaizen.com/insights/lean-manufacturing-kanban/>

- [15] HuggingFace. (2024). HuggingFace Inference API Documentation.
<https://huggingface.co/docs/api-inference>
- [16] FastAPI Documentation. (2024). FastAPI — Modern, Fast Web Framework for Python.
<https://fastapi.tiangolo.com/>
- [17] Google DeepMind. (2024). Gemma: Open Models Based on Gemini Research and Technology. <https://ai.google.dev/gemma>
- [18] Pydantic Documentation. (2024). Pydantic — Data Validation Using Python Type Hints.
<https://docs.pydantic.dev/>